

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación

Carrera de Software
Asignatura: Aplicaciones Web

BIOPET

Sistema web de gestión veterinaria

Informe académico final — Entrega Final v1.0.0

Equipo **BMT**

Mariscal Cabrera Jaime Josué	—	jmariscalc@uteq.edu.ec
Beltrán Montiel Fred Adrián	—	fbeltranm@uteq.edu.ec
Taípe Mora Zaida Melissa	—	ztaipem@uteq.edu.ec

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
gguerrero@uteq.edu.ec

Período académico: Presencial 2026 – 2027 — Curso/paralelo: A

DOI del software (Zenodo)	10.5281/zenodo.21988746
DOI del <i>dataset</i> de evidencias (Zenodo)	10.5281/zenodo.21988785
Repositorio	github.com/Grinjoww/ENTREGA-FINAL-BIOPET
Imagen de contenedor (GHCR, digest inmutable)	sha256:ef1e857a...6bef5163
Frontend (producción)	https://biopet-frontend.onrender.com
Backend (producción)	https://biopet-backend-dh5e.onrender.com

Quevedo, 18 de agosto de 2026

Índice general

Resumen	1
Abstract	2
1. Introducción	3
1.1. Contexto: gestión veterinaria y digitalización incompleta	3
1.2. Problema abordado	3
1.3. Motivación	4
1.4. Preguntas de investigación	4
1.5. Objetivo general	4
1.6. Alcance	5
1.7. Contribución del proyecto	5
1.8. Organización de este informe	5
2. Marco teórico	6
2.1. Ingeniería de requisitos	6
2.2. Arquitectura de software: el modelo C4	6
2.3. Calidad de software: ISO/IEC 25010	6
2.4. Arquitectura REST y JWT	7
2.5. Seguridad de aplicaciones web: OWASP Top 10	7
2.6. Acceso a datos: JPA y procedimientos almacenados	7
2.7. Patrón <i>cache-aside</i>	8
2.8. Síntesis	8
3. Trabajos relacionados	9
3.1. Criterios de elegibilidad y fuentes	9
3.2. Estudios incluidos	10
3.2.1. Pacheco, García & Reyes (2018) — Técnicas de elicitación de requisitos	10
3.2.2. Bangor, Kortum & Miller (2008) — Benchmark empírico del SUS .	10
3.2.3. Cedeño Ochoa, Catuto Murillo & Rodas-Silva (2021) — Aplicaciones web para clínicas veterinarias en Ecuador	10
3.2.4. Rodríguez, Llerena, Guevara, Baren & Castro (2024) — OSCRUM	11
3.2.5. Putri, Hadi & Ramdani (2017) — Pruebas de rendimiento de un sistema web académico	11
3.2.6. Yang et al. (2026) — Vulnerabilidades en implementaciones de JWT (NDSS)	11
3.2.7. Estdale & Georgiadou (2018) — Aplicación práctica de ISO/IEC 25010	11
3.2.8. Bucao, Quinones, Abong, Penuela & Sun (2023) — VETelgeuse . .	12
3.2.9. Llaneta, Guelas, Labanan, Mercado & Sasis (2022) — TerraVet . .	12

3.2.10. Zmaranda, Pop-Fele, Gyorödi, Gyorödi & Pecherle (2020) — CRUD con ORM vs. SQL directo	12
3.3. Síntesis comparativa	12
3.4. Limitaciones de esta revisión	13
4. Metodología	14
4.1. Enfoque metodológico	14
4.2. Design Science Research: las seis actividades de Peffers et al.	15
4.3. Goal–Question–Metric (GQM)	15
4.4. Diseño de evaluación	16
4.5. Reproducibilidad	17
4.6. Amenazas a la validez (resumen metodológico)	17
5. Ingeniería de requisitos	18
5.1. Priorización MoSCoW	18
5.2. Requisitos funcionales implementados y verificados	18
5.2.1. Requisitos incorporados en la Unidad IV (REQ-F-023 a REQ-F-025)	18
5.3. Requisitos pendientes, heredados de la Entrega 1A	19
5.4. Requisitos no funcionales relevantes	21
5.5. Trazabilidad y estado de la validación automática	22
5.6. Marco INCOSE	22
5.7. Cambios entre versiones	22
6. Diseño y arquitectura	23
6.1. Arquitectura general	23
6.2. C4 Nivel 2 — Contenedores	23
6.3. C4 Nivel 3 — Componentes del backend	24
6.4. Acceso JPA formal a procedimientos almacenados	24
6.5. Decisiones de arquitectura relevantes (ADR)	26
6.6. Patrones y decisiones de seguridad relevantes al diseño	27
6.7. ISO/IEC 25010: marco de calidad	27
7. Implementación	28
7.1. Autenticación y autorización	28
7.2. Gestión de usuarios	28
7.3. Gestión de mascotas	28
7.4. Citas	28
7.5. Consultas	29
7.6. Vacunas	29
7.7. Integración con API externa	29
7.8. Caché (patrón cache-aside)	29
7.9. Procedimientos almacenados	30
7.10. Migraciones Flyway	30
7.11. Manejo uniforme de errores (RFC 7807)	31
7.12. Frontend	31
7.13. Persistencia	32

8. Pruebas y calidad	33
8.1. Pruebas automatizadas del backend	33
8.2. Cobertura JaCoCo	33
8.3. Rendimiento (k6)	34
8.4. Usabilidad (SUS)	35
8.5. Calidad web automática (Lighthouse)	36
8.6. Diferenciación de los tipos de medición	37
9. Seguridad	38
9.1. Autenticación por cookies y revocación	38
9.2. Autorización por rol y por propiedad	38
9.3. Cabeceras de seguridad y TLS	39
9.4. Auditoría OWASP Top 10 (resumen)	40
9.5. Protección estructural contra inyección (A03)	40
9.6. Auditoría estática: SpotBugs + Find Security Bugs	40
9.7. Auditoría dinámica: OWASP ZAP Baseline	41
9.8. Auditoría de SQL dinámico en procedimientos almacenados	41
9.9. Postman	42
9.10. Limitaciones reales de este capítulo	42
10.Despliegue y reproducibilidad	43
10.1. Despliegue real en producción	43
10.2. Continuous Integration	43
10.3. GHCR (imagen de contenedor del backend)	44
10.4. Archivado permanente en Zenodo	44
10.5. Pendientes honestos de publicación	45
10.6. Reproducibilidad de punta a punta	47
11.Matriz de trazabilidad	48
12.Resultados	52
12.1. Qué se construyó	52
12.2. Qué se validó y con qué resultado	53
12.3. Qué requisitos fueron satisfechos	53
12.4. Evidencia que respalda cada resultado	53
12.5. Consistencia numérica frente a entregas previas	54
13.Discusión	55
13.1. Cobertura y calidad de código	55
13.2. Rendimiento	55
13.3. Seguridad	55
13.4. Usabilidad	56
13.5. Calidad	56
13.6. Despliegue y reproducibilidad	56
13.7. Comparación con trabajos relacionados: límites de la afirmación	56
13.8. Respuesta explícita a las preguntas de investigación	57

14.Amenazas a la validez y limitaciones	58
14.1. Validez interna	58
14.2. Validez externa	58
14.3. Validez de constructo	58
14.4. Validez de conclusión	59
14.5. Limitación temporal explícita: operación sostenida no verificable hoy . . .	59
14.6. Otras limitaciones del estudio	59
15.Conclusiones	61
15.1. Principales logros	61
15.2. Evidencia	61
15.3. Valor del proyecto	61
15.4. Estado de release	62
15.5. Trabajo futuro	62
16.Declaraciones	63
16.1. Contribuciones (taxonomía CRediT)	63
16.2. Autores únicos de esta entrega	63
16.3. Ética	64
16.4. Disponibilidad de datos	64
16.5. Disponibilidad del código	64
16.6. Conflicto de intereses	64
16.7. Reproducibilidad	64
17.Anexos	65
17.1. Glosario de siglas	65
17.2. Endpoints principales de la API	66
17.3. Documentos de referencia en el repositorio	66
17.4. Clases de prueba automatizadas citadas en este informe	67
17.5. Anexo A — Bitácora de observaciones docentes	67
17.6. Anexo B — Estrategia de búsqueda PRISMA	68
17.7. Anexo C — Instrumento SUS aplicado	68
17.8. Anexo D — Evidencia de integración continua	68
17.9. Anexo E — Checklist Ralph et al. (Engineering Research)	69
17.10Anexo F — Matriz de trazabilidad completa	69
17.11Anexo G — Clasificación de referencias por nivel de impacto	73
17.12Evidencias pendientes de incorporación como figura	74

Índice de figuras

3.1.	Diagrama de flujo PRISMA de identificación y selección de estudios (regenerado 2026-08-18). Bloque Zaida completamente cuantificado (15 identificados → 12 excluidos → 3 incluidos); bloque Fred completamente cuantificado (13 candidatos por DOI → 2 excluidos → 3 incluidos); bloque Jaime con el conteo final de incluidos (4), sin registro de candidatos excluidos (único gap declarado que persiste en el diagrama, señalado con línea punteada).	10
6.1.	Diagrama C4 Nivel 2 (contenedores) de BIOPET: frontend, backend, PostgreSQL y Redis/Valkey, con los protocolos reales entre ellos.	23
6.2.	Diagrama C4 Nivel 3 (componentes) del backend de BIOPET.	24
6.3.	Acceso híbrido a procedimientos almacenados: invocación formal JPA sobre las rutinas reclasificadas como PROCEDURE	25
6.4.	Invocación real de una rutina almacenada verificada desde Postman.	26
7.1.	Pantalla de inicio de sesión del frontend de BIOPET.	32
7.2.	CRUD de mascotas en el frontend: listado, alta y edición.	32
8.1.	Salida real de <code>mvn clean verify</code> : pruebas del backend con BUILD SUCCESS	33
8.2.	Resumen navegable de JaCoCo con la cobertura por paquete.	34
8.3.	Salida de consola de una corrida de k6 en caliente.	35
8.4.	Salida de consola de una corrida de k6 en frío.	35
8.5.	Resultados agregados del instrumento SUS (n=18): media, desviación típica, IC 95 %.	36
9.1.	Respuesta 401 (ProblemDetail) capturada desde la colección Postman de BIOPET.	38
9.2.	Salida de <code>openssl s_client -tls1_3: TLSv1.3 y TLS_AES_256_GCM_SHA384</code> negociados (entorno de desarrollo).	39
9.3.	Cabeceras de seguridad reales capturadas con <code>curl.exe -i</code>	39
10.1.	Migraciones Flyway (V1–V6) aplicadas en orden desde una base vacía.	45
10.2.	Servicios de <code>docker compose</code> en estado healthy tras <code>make up</code>	46
10.3.	Imágenes de terceros fijadas por <code>digest sha256</code> en <code>docker-compose.yml</code>	46

Índice de cuadros

3.1. Síntesis comparativa de los diez estudios incluidos frente a BIOPET.	12
4.1. Mapeo de BIOPET sobre las seis actividades de Peffers et al. [1].	15
4.2. Preguntas y métricas GQM, con el valor de evidencia vigente al cierre de la Entrega Final.	16
5.1. Requisitos funcionales pendientes o parciales heredados de la Entrega 1A (fuente: docs/requisitos/SRS.md).	19
5.2. Requisitos no funcionales relevantes citados en este informe (fuente: docs/requisitos/SRS.md, sección 3.2).	21
6.1. Catálogo de las seis rutinas almacenadas de BIOPET (fuente: docs/basedatos/CATALOGOSP.md).	
6.2. ADR vigentes del repositorio.	26
6.3. Mapeo resumido de ISO/IEC 25010 sobre evidencia de BIOPET.	27
8.1. Resultado real de la suite de pruebas del backend.	33
8.2. Cobertura medida por jacoco:check (fuente: docs/mediciones/jacoco/METRICS.md).	34
8.3. Resultado real de las diez corridas de k6 (fuente: docs/mediciones/perf/REPORT.md).	34
8.4. Resultados agregados del instrumento SUS (n = 18).	36
8.5. Resultado real de la corrida Lighthouse del 2026-08-18 (12 corridas, mobile+desktop; fuente: docs/mediciones/lighthouse/lhci-20260818-0538-*.json).	37
9.1. Resultado de la auditoría manual OWASP por control (fuente: docs/mediciones/sec/REPORT.md).	40
9.2. Resumen del análisis estático (fuente: docs/mediciones/sec/static-analysis/README.md).	40
9.3. Resumen ZAP Baseline (fuente: docs/mediciones/sec/zap/README.md).	41
10.1. Servicios reales de producción (fuente: docs/despliegue/DEPLOYMENT.md).	43
10.2. Jobs del pipeline de CI.	44
10.3. Publicación GHCR (fuente: docs/publicacion/PAQUETE-V1.0.0.md).	44
10.4. DOI publicados en Zenodo.	45
11.1. Matriz de trazabilidad (selección representativa, v1.0.0). Matriz completa: docs/trazabilidad/matriz.csv.	49
12.1. Síntesis de resultados cuantitativos de BIOPET v1.0.0.	53
16.1. Resumen de contribuciones CRediT por integrante (detalle completo en CONTRIBUTORS.md).	63

17.1. Siglas usadas en este informe.	65
17.2. Endpoints reales expuestos por el backend v1.0.0.	66
17.3. Observaciones docentes acumuladas y su estado de cierre.	67
17.4. Resumen del checklist Ralph et al. (2021), estándar Engineering Research.	69
17.5. Matriz de trazabilidad completa (fuente: docs/trazabilidad/matriz.csv).	70
17.6. Clasificación de las 41 referencias del informe.	73

Índice de listados

7.1. Patrón <i>cache-aside</i> declarativo en <code>MascotaService.java</code> (fragmento real).	29
7.2. Acceso JPA formal a procedimientos almacenados en <code>ProcedimientoBiopetRepository.java</code> (fragmento real).	30
7.3. Fábrica de errores RFC 7807 en <code>ProblemDetailFactory.java</code> (fragmento real).	31

Resumen

Problema. La mayoría de clínicas veterinarias, en particular en contextos como Ecuador, siguen operando con registros manuales o en papel, lo que provoca pérdida y duplicidad de expedientes, dificultad para consultar el historial de una mascota y ausencia de indicadores de gestión básicos [2]. **Solución.** BIOPET es un sistema web de gestión veterinaria (backend Spring Boot 3.2 / Java 21, frontend Angular 17, PostgreSQL 18 en producción, caché Redis/Valkey 8) que cubre autenticación y autorización por rol y por propiedad, un CRUD completo de mascotas, y los módulos de citas, consultas y vacunas incorporados durante la Unidad IV, respaldados por seis rutinas almacenadas de PostgreSQL con acceso JPA formal (@Procedure/@NamedStoredProcedureQuery). **Metodología.** El proyecto se enmarca como *Engineering Research/Design Science*, siguiendo las seis actividades de Peffers et al. [1] y un marco Goal–Question–Metric [3] para vincular cada afirmación cuantitativa con una métrica y una fuente de evidencia real, sin diseño experimental controlado ni afirmación de validación industrial. **Principales validaciones.** 205 pruebas automatizadas del backend (0 fallos, 0 errores); cobertura JaCoCo de 91,80 % en líneas y 79,39 % en ramas sobre un umbral automático del 70 %; diez corridas de rendimiento con k6 (cinco en frío, cinco en caliente) con 0,0 % de tasa de error; usabilidad medida con el instrumento System Usability Scale (SUS) sobre 18 participantes reales (media 74,44/100, IC 95 % [63,33, 85,56]); auditoría de seguridad OWASP Top 10 con evidencia real, escaneo dinámico OWASP ZAP Baseline (0 alertas de severidad alta) y análisis estático Spot-Bugs/Find Security Bugs (0 hallazgos relacionados con inyección SQL); y despliegue real en producción (Render, HTTPS activo, /actuator/health en estado UP). El software y el conjunto de datos de evidencias empíricas están archivados de forma permanente en Zenodo con DOI reales (10.5281/zenodo.21988746 y 10.5281/zenodo.21988785 respectivamente), y la imagen del backend está publicada en GitHub Container Registry con una referencia inmutable por *digest sha256*. **Conclusión principal.** BIOPET demuestra, con evidencia reproducible y trazable en todas sus dimensiones de calidad (funcionalidad, seguridad, rendimiento, usabilidad, mantenibilidad y portabilidad, enmarcadas en ISO/IEC 25010 [4]), que es posible construir y evaluar empíricamente, dentro del alcance de un Proyecto Fin de Curso, un artefacto de software funcional, seguro según los controles auditados y reproducible de punta a punta; las limitaciones de alcance (entorno académico, tamaño muestral, ausencia de comparación con sistemas veterinarios alternativos) se documentan sin ocultarlas en el capítulo de amenazas a la validez.

Palabras clave: gestión veterinaria; aplicaciones web; Spring Boot; Angular; JWT; PostgreSQL; ISO/IEC 25010; usabilidad (SUS); seguridad OWASP; reproducibilidad.

Abstract

Problem. Most veterinary clinics, particularly in contexts such as Ecuador, still rely on manual or paper-based records, causing lost and duplicated files, difficulty retrieving a pet's clinical history, and the absence of basic management indicators [2]. **Solution.** BIOPET is a web-based veterinary management system (Spring Boot 3.2/Java 21 backend, Angular 17 frontend, PostgreSQL 18 in production, Redis/Valkey 8 cache) covering role- and ownership-based authentication and authorization, a complete pet CRUD, and the appointments, consultations and vaccines modules added during Unit IV, backed by six PostgreSQL stored routines with formal JPA access (`@Procedure/@NamedStoredProcedureQuery`). **Methodology.** The project follows an Engineering Research/Design Science approach, mapped onto the six Peffers et al. [1] activities, using a Goal–Question–Metric [3] framework so every quantitative claim is tied to a metric and a verifiable evidence source, with no controlled experimental design and no claim of industrial validation. **Main validations.** 205 automated backend tests (0 failures, 0 errors); JaCoCo coverage of 91.80 % lines and 79.39 % branches against an automatic 70 % gate; ten k6 performance runs (five cold, five warm) with 0.0 % error rate; usability measured with the System Usability Scale (SUS) on 18 real participants (mean 74.44/100, 95 % CI [63,33, 85,56]); an OWASP Top 10 security audit with real evidence, a dynamic OWASP ZAP Baseline scan (0 high-severity alerts) and a SpotBugs/Find Security Bugs static analysis (0 SQL-injection-related findings); and a real production deployment (Render, HTTPS enabled, `/actuator/health` reporting UP). The software and the empirical evidence dataset are permanently archived on Zenodo with real DOIs (10.5281/zenodo.21988746 and 10.5281/zenodo.21988785 respectively), and the backend image is published on the GitHub Container Registry with an immutable `sha256` digest reference. **Main conclusion.** BIOPET shows, with reproducible and traceable evidence across every quality dimension (functionality, security, performance, usability, maintainability and portability, framed under ISO/IEC 25010 [4]), that it is possible to build and empirically evaluate, within the scope of a capstone course project, a functional software artifact that is secure with respect to the audited controls and reproducible end to end; scope limitations (academic environment, sample size, absence of comparison against alternative veterinary systems) are documented without concealment in the threats-to-validity chapter.

Keywords: veterinary management; web applications; Spring Boot; Angular; JWT; PostgreSQL; ISO/IEC 25010; usability (SUS); OWASP security; reproducibility.

Capítulo 1

Introducción

1.1. Contexto: gestión veterinaria y digitalización incompleta

La administración de una clínica veterinaria involucra procesos que, en ausencia de una herramienta digital centralizada, dependen de registros manuales o en hojas de cálculo dispersas: identificación de mascotas y dueños, historial de atenciones, control de vacunas y programación de citas. La literatura reciente sobre el contexto ecuatoriano confirma que esta carencia no es anécdota: Cedeño Ochoa, Catuto Murillo & Rodas-Silva documentan explícitamente cómo la mayoría de clínicas veterinarias locales carecen de herramientas tecnológicas de gestión, lo que provoca pérdida y duplicidad de expedientes por manejo manual [2]. Rodríguez et al. documentan un patrón similar en otro contexto geográfico, motivando un sistema de código abierto para el mismo dominio [5]. BIOPET nace de ese mismo diagnóstico: la necesidad de un sistema web que centralice identificación, autenticación, y la gestión de mascotas, citas, consultas y vacunas de una clínica veterinaria, con evidencia empírica de que efectivamente funciona y es razonablemente seguro, no solo de que fue programado.

1.2. Problema abordado

El problema que BIOPET aborda tiene dos capas. La primera es funcional: proveer una plataforma web que permita registrar usuarios con distintos roles (administrador, veterinario, auxiliar, dueño), administrar mascotas asociadas a un dueño, y gestionar el ciclo de atención veterinaria (citas, consultas, vacunas) con control de acceso adecuado a cada rol. La segunda capa, menos visible pero igual de crítica, es de **evidencia empírica**: un sistema académico puede afirmar que “funciona” sin que esa afirmación esté respaldada por pruebas automatizadas, medición de rendimiento bajo carga, auditoría de seguridad real o percepción de usabilidad medida con un instrumento estandarizado. BIOPET trata esa segunda capa como parte del problema mismo a resolver, no como un anexo opcional: cada afirmación cuantitativa de este informe cita su fuente de evidencia real en el repositorio.

1.3. Motivación

Tres entregas previas (Entrega 1A, Entrega 1B y Tercera Entrega) generaron retroalimentación docente real, registrada íntegramente en `docs/observaciones/OBSERVACIONES.md`: 15 observaciones formales, todas cerradas a la fecha de este informe, cubriendo desde la ausencia de un repositorio accesible hasta la falta de evidencia Lighthouse o de un DOI de archivado permanente. Esa bitácora de correcciones es, en sí misma, parte de la motivación de este documento: la Entrega Final no reescribe el proyecto desde cero, sino que consolida el trabajo ya evaluado, corrigiendo explícitamente lo señalado y documentando con honestidad lo que sigue pendiente.

1.4. Preguntas de investigación

Este informe organiza su evaluación empírica alrededor de tres preguntas de investigación, cada una medible con evidencia real ya recolectada y respondida explícitamente en el capítulo 13 (sección 13.8):

- RQ1.** ¿En qué medida BIOPET satisface los requisitos funcionales y no funcionales definidos para la Entrega Final, y cuáles permanecen explícitamente pendientes?
- RQ2.** ¿Qué resultados presenta BIOPET en cobertura de pruebas, rendimiento, usabilidad, calidad web y seguridad bajo las evaluaciones empíricas realmente ejecutadas?
- RQ3.** ¿En qué medida el proceso de despliegue, integración continua, publicación de la imagen de contenedor y archivado permanente permiten reproducir y verificar el artefacto final de forma independiente?

1.5. Objetivo general

Consolidar, evaluar empíricamente y documentar académicamente el sistema BIOPET en su versión v1.0.0, integrando evidencia real de calidad funcional, seguridad, rendimiento, usabilidad, mantenibilidad y reproducibilidad, enmarcada en el modelo de calidad ISO/IEC 25010 y en una metodología de investigación de ingeniería explícita, sin inventar resultados ni ocultar limitaciones.

Objetivos específicos

1. Verificar el cumplimiento de los requisitos funcionales y no funcionales declarados en el SRS y documentar la arquitectura del sistema (incluidos los módulos de Unidad IV), distinguiendo explícitamente lo verificado de lo pendiente. (*RQ1*)
2. Reportar la evidencia empírica real de pruebas automatizadas, cobertura, rendimiento, usabilidad y calidad web recolectada hasta el cierre de esta entrega. (*RQ2*)
3. Auditar la seguridad del sistema desde tres ángulos complementarios (revisión manual OWASP, análisis dinámico ZAP, análisis estático SpotBugs/Find Security Bugs), incluyendo el tratamiento explícito del único hallazgo de severidad alta detectado. (*RQ2*)
4. Documentar el despliegue real en producción, la publicación de la imagen de contenedor en GHCR, el archivado permanente en Zenodo y la reproducibilidad de punta a punta del artefacto. (*RQ3*)

5. Discutir los resultados frente a la literatura relacionada disponible y declarar honestamente las amenazas a la validez y las limitaciones del estudio, incluyendo condiciones aún no verificables (por ejemplo, operación sostenida durante períodos prolongados). (*RQ1–RQ3*)

1.6. Alcance

El alcance funcional implementado y verificado cubre: registro y autenticación de usuarios por cookies seguras; autorización por rol y por propiedad; CRUD completo de mascotas; y los módulos de citas, consultas y vacunas incorporados en la Unidad IV, respaldados por seis rutinas almacenadas de PostgreSQL. Quedan explícitamente **fuera** del alcance implementado, y así se documentan en el capítulo de requisitos (capítulo 5): historial clínico cronológico completo como módulo independiente, integración con dispositivos IoT, facturación digital, reportes exportables en PDF/Excel, y recomendaciones clínicas generadas por IA externa. El alcance de evaluación empírica cubre pruebas automatizadas, cobertura, rendimiento, usabilidad, calidad web, y seguridad estática/dinámica, todas ejecutadas en un entorno académico (desarrollo local y despliegue en el plan gratuito de Render), no en un entorno industrial de alta disponibilidad.

1.7. Contribución del proyecto

La contribución principal de BIOPET, frente al panorama de trabajos relacionados revisado en el capítulo 3, no es una técnica nueva de ingeniería de software, sino la **amplitud verificable de la evidencia empírica** recolectada sobre un mismo artefacto real: cobertura de pruebas, rendimiento bajo carga, usabilidad medida con instrumento estandarizado, calidad web automatizada, y seguridad auditada por tres métodos distintos, todo trazable a requisitos formales y reproducible con comandos documentados (**make up**, **mvn clean verify**, scripts de **scripts/**). Ningún trabajo relacionado revisado reporta las seis dimensiones a la vez sobre el mismo sistema (sección 3.3).

1.8. Organización de este informe

El resto del documento se organiza así: el capítulo 2 resume el marco teórico y normativo usado en todo el informe; el capítulo 3 revisa los trabajos relacionados bajo un protocolo inspirado en PRISMA 2020; el capítulo 4 describe la metodología de investigación de ingeniería adoptada; el capítulo 5 documenta la ingeniería de requisitos; el capítulo 6 la arquitectura y las decisiones de diseño; el capítulo 7 los módulos implementados; los capítulos 8 y 9 la evidencia de pruebas/calidad y de seguridad; el capítulo 10 el despliegue y la reproducibilidad; el capítulo 11 la matriz de trazabilidad; los capítulos 12 y 13 integran y discuten los resultados; el capítulo 14 documenta las amenazas a la validez y las limitaciones; el capítulo 15 presenta las conclusiones; el capítulo 16 recoge las declaraciones formales (CRediT, ética, disponibilidad); y los anexos (capítulo 17) complementan con glosario, endpoints y documentos de referencia.

Capítulo 2

Marco teórico

Este capítulo reúne, de forma compacta, los conceptos y normas que sustentan las decisiones técnicas y metodológicas descritas en el resto del informe. No repite el detalle de aplicación de cada concepto a BIOPET (eso corresponde a los capítulos 5 a 10); se limita a definir el marco conceptual usado y a señalar dónde se aplica.

2.1. Ingeniería de requisitos

La especificación de requisitos de BIOPET sigue la estructura de ISO/IEC/IEEE 29148:2018, estándar internacional que define el contenido mínimo de un documento de Especificación de Requisitos de Software (SRS): descripción global del producto, requisitos funcionales y no funcionales enunciados de forma verificable, y trazabilidad explícita hacia artefactos derivados (historias de usuario, casos de uso, pruebas). La redacción individual de cada requisito sigue además las características de calidad de la *INCOSE Guide to Writing Requirements* [6] — en particular *Unambiguous* (un requisito admite una única interpretación) y *Verifiable* (existe un método objetivo para confirmar su cumplimiento) — aplicadas en el capítulo 5. Las historias de usuario y los casos de uso son las dos técnicas complementarias de elicitación/especificación usadas para descomponer cada requisito en un flujo de interacción concreto, según la práctica comparada por Pacheco, García & Reyes [7] (capítulo 3).

2.2. Arquitectura de software: el modelo C4

El modelo C4 de Brown [8] describe la arquitectura de un sistema en niveles de abstracción progresivos (Contexto, Contenedores, Componentes, Código), cada uno dirigido a una audiencia distinta. BIOPET documenta formalmente los niveles de Contenedores y Componentes (capítulo 6, secciones 6.2 y 6.3), eligiendo ese nivel de detalle porque es el que permite razonar sobre las decisiones de despliegue y de diseño interno del backend sin caer en el detalle de clases individuales, que cambia con mayor frecuencia que la arquitectura.

2.3. Calidad de software: ISO/IEC 25010

ISO/IEC 25010:2011 [4] define un modelo de calidad de producto de software organizado en ocho características (Funcionalidad, Eficiencia de desempeño, Compatibilidad,

Usabilidad, Confiabilidad, Seguridad, Mantenibilidad, Portabilidad), cada una con subcaracterísticas específicas. Se usa en este informe como vocabulario común para organizar la evidencia empírica recolectada (capítulo 8), siguiendo el enfoque de aplicación práctica descrito por Estdale & Georgiadou [9]: mapear cada característica contra evidencia real, declarando explícitamente cuándo una subcaracterística no fue evaluada, en vez de omitirla silenciosamente (sección 6.7).

2.4. Arquitectura REST y JWT

La API de BIOPET sigue el estilo arquitectónico REST (*Representational State Transfer*) sobre HTTP: recursos identificados por URL, verbos HTTP con semántica definida, y respuestas sin estado del lado del servidor entre solicitudes. La autenticación usa JSON Web Token (JWT), formato compacto y autocontenido definido en RFC 7519 [10]: un token firmado digitalmente que transporta un conjunto de *claims* (declaraciones) verificables sin necesidad de una consulta adicional al servidor. BIOPET usa JWT firmado con HMAC-SHA256, transportado en cookies en vez de en el cuerpo de la respuesta o en `localStorage` (capítulo 9), decisión de arquitectura registrada en ADR-006. Los errores de la API siguen RFC 7807 [11] (*Problem Details for HTTP APIs*), formato estándar para comunicar fallos HTTP de forma estructurada y legible por máquina.

2.5. Seguridad de aplicaciones web: OWASP Top 10

El OWASP Top 10 [12] es la clasificación de referencia de la industria para los riesgos de seguridad más críticos en aplicaciones web (control de acceso roto, fallas criptográficas, inyección, configuración de seguridad, fallas de identificación/autenticación, fallas de registro/monitoreo, entre otros). BIOPET usa esta clasificación como marco de auditoría manual (capítulo 9), complementada con análisis dinámico (OWASP ZAP) y estático (SpotBugs + Find Security Bugs), siguiendo una lógica de defensa en profundidad: ningún método único de auditoría se considera suficiente por sí solo.

2.6. Acceso a datos: JPA y procedimientos almacenados

Spring Data JPA (*Java Persistence API*) provee un mapeo objeto-relacional (ORM) que traduce operaciones sobre entidades Java a sentencias SQL generadas automáticamente, adecuado para operaciones CRUD simples sobre una única tabla. Para operaciones que involucran lógica de negocio, validación cruzada entre tablas o agregaciones complejas, BIOPET usa procedimientos y funciones almacenados de PostgreSQL, invocados desde el backend mediante `@Procedure/@NamedStoredProcedureQuery` de Spring Data JPA (sección 6.4). Esta estrategia híbrida sigue la misma lógica documentada por Zmaranda et al. [13]: dejar que cada mecanismo de acceso a datos resuelva el tipo de operación para el que fue diseñado, en vez de forzar toda la lógica a través de un único mecanismo. Esta decisión es consistente con evidencia empírica comparativa adicional: SQL nativo supera en tiempo de ejecución a las consultas generadas por ORM tanto en C# [14] como, vía JDBC directo, en Java [15], y la implementación de la capa de persistencia tiene un impacto medible sobre el rendimiento general de la aplicación [16]. La comparación de

rendimiento entre pilas REST equivalentes (Spring Boot y .NET Core), relevante para la decisión de arquitectura registrada en ADR-002, sigue también una metodología de *benchmarking* comparable a la usada aquí [17, 18].

2.7. Patrón *cache-aside*

El patrón *cache-aside* (también llamado *lazy loading*) consiste en que la aplicación consulta primero la caché; si el dato no está presente (*cache miss*), lo obtiene del almacén de datos primario y lo escribe en la caché para lecturas futuras; en cada escritura sobre el dato primario, la entrada de caché correspondiente se invalida. BIOPET implementa este patrón de forma declarativa con las anotaciones `@Cacheable` (lectura) y `@CacheEvict` (invalidación) de Spring sobre Redis/Valkey (sección 8.3, listado 7.1), respaldado por trabajos previos sobre la efectividad de esta estrategia en acceso a datos relacionales [19]. La configuración del tiempo de vida (TTL) de cada entrada de caché, externalizada por variable de entorno en BIOPET (`CACHE_TTL_MS`), sigue la misma recomendación de análisis a gran escala de clústeres de caché en memoria: la configuración del TTL es un factor clave del comportamiento observado [20]. El despliegue contenerizado de BIOPET (capítulo 10) tiene, como cualquier despliegue en contenedores, un *overhead* medible según su configuración [21].

2.8. Síntesis

Los conceptos anteriores no se aplican de forma aislada: la ingeniería de requisitos (29148/INCOSE) alimenta la matriz de trazabilidad (capítulo 11); el modelo C4 documenta la arquitectura que implementa esos requisitos; ISO/IEC 25010 organiza la evidencia empírica que verifica si esa arquitectura cumple sus atributos de calidad; y REST/JWT/OWASP/JPA/*cache-aside* son las decisiones de implementación concretas evaluadas por esa evidencia. Este marco común es lo que permite, en el capítulo 13, discutir los resultados de BIOPET en un mismo vocabulario en vez de como hallazgos inconexos.

Capítulo 3

Trabajos relacionados

Este capítulo integra la revisión de trabajos relacionados realizada por Zaida (bloque requisitos/usabilidad/dominio veterinario), Jaime (bloque arquitectura/seguridad/rendimiento/calidad) y Fred (bloque arquitectura de datos/rendimiento), consolidada y cerrada con un protocolo inspirado en PRISMA 2020 [22] en `docs/checklists/prisma2020.md`. PRISMA fue diseñado para revisiones sistemáticas de intervenciones sanitarias; aquí se adapta explícitamente como marco de **rigor y transparencia** para una revisión narrativa de trabajos relacionados de ingeniería de software, no como un metaanálisis clínico — varios ítems orientados a estadística de efectos clínicos se marcan *No aplica* en el checklist original en vez de forzarlos.

3.1. Criterios de elegibilidad y fuentes

Inclusión: estudio con DOI o enlace verificable; publicado en fuente académica o en una revista/*proceedings* con revisión por pares (o norma/documento oficial de industria sin DOI, como ISO u OWASP); relevante a al menos uno de los tres bloques temáticos (requisitos, usabilidad o dominio veterinario — Zaida; arquitectura, seguridad, rendimiento o calidad — Jaime; arquitectura de datos/rendimiento — Fred). **Exclusión:** blogs comerciales sin revisión por pares, páginas de venta de software, trabajos sin autor o fecha verificable. **Fuentes consultadas:** IEEE Xplore, ACM Digital Library, Scopus y Google Scholar (bloque Zaida, búsqueda 2026-08-17); auditoría de la bibliografía ya existente más verificación dirigida contra Crossref y páginas oficiales de editorial/conferencia/norma (bloque Jaime, misma fecha); verificación dirigida de DOI reales, uno a uno, con un filtro temporal explícito de 2020–2026 (bloque Fred, 2026-08-17). El proceso de selección fue independiente por bloque (cada autor filtró su propio tema), sin doble revisor cruzado por estudio — limitación declarada explícitamente, no oculta.

De un total de 15 candidatos revisados por Zaida (12 excluidos por no cumplir el criterio de elegibilidad), una verificación dirigida de 4 candidatos por Jaime, y 13 candidatos por DOI verificados uno a uno por Fred (2 descartados: uno por fuera del rango temporal 2020–2026, otro por fuente inaccesible con HTTP 403), el conjunto final incluye **10 estudios**, cumpliendo la meta original de $\geq 8-9$. Los tres bloques (Zaida, Jaime, Fred) están integrados en el checklist PRISMA cerrado (`docs/checklists/prisma2020.md`, cierre 2026-08-18) y en el diagrama de flujo regenerado que se muestra en la figura 3.1.

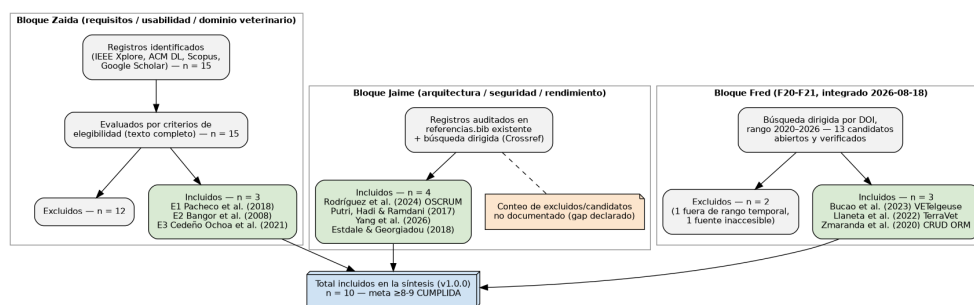


Figura 3.1: Diagrama de flujo PRISMA de identificación y selección de estudios (regenerado 2026-08-18). Bloque Zaida completamente cuantificado (15 identificados → 12 excluidos → 3 incluidos); bloque Fred completamente cuantificado (13 candidatos por DOI → 2 excluidos → 3 incluidos); bloque Jaime con el conteo final de incluidos (4), sin registro de candidatos excluidos (único gap declarado que persiste en el diagrama, señalado con línea punteada).

3.2. Estudios incluidos

3.2.1. Pacheco, García & Reyes (2018) — Técnicas de elicitación de requisitos

Revisión sistemática sobre 140 estudios (1993–2015) que identifica las entrevistas estructuradas como la técnica de elicitación de requisitos con mayor evidencia empírica de madurez y efectividad [7]. El SRS de BIOPET (`docs/requisitos/SRS.md`) documenta requisitos ya elicitados (historias de usuario, casos de uso, patrón ISO/IEC/IEEE 29148) pero no declara explícitamente qué técnica se usó para llegar a ellos; a diferencia de Pacheco et al., que comparan técnicas entre sí, BIOPET aplica una técnica ya elegida y la documenta con trazabilidad formal completa ($\text{SRS} \rightarrow \text{HU} \rightarrow \text{CU} \rightarrow \text{matriz}$), sin reportar esa comparación.

3.2.2. Bangor, Kortum & Miller (2008) — Benchmark empírico del SUS

Analiza aproximadamente diez años de datos SUS sobre cientos de productos, estableciendo un puntaje medio de referencia de la industria de 68/100 (“sobre el promedio”) y 80,3 como umbral de “excelente” [23]. Este estudio es la fuente del benchmark contra el que se interpreta el puntaje SUS propio de BIOPET (sección 8.4): BIOPET no construye un benchmark nuevo, usa el ya existente para interpretar una medición real sobre un sistema concreto.

3.2.3. Cedeño Ochoa, Catuto Murillo & Rodas-Silva (2021) — Aplicaciones web para clínicas veterinarias en Ecuador

Caso de estudio sobre el desarrollo de una aplicación web para gestión de una clínica veterinaria ecuatoriana, documentando la mejora cualitativa de procesos administrativos al migrar de manejo manual a una plataforma centralizada [2]. Es el trabajo con contexto país más cercano encontrado y verificado. A diferencia de BIOPET, no reporta una evaluación

empírica multimétrica del sistema resultante (sin cobertura de pruebas, rendimiento bajo carga, seguridad auditada ni usabilidad medida con instrumento estandarizado).

3.2.4. **Rodríguez, Llerena, Guevara, Baren & Castro (2024) — OSCRUM**

Estudio de caso sobre el desarrollo de SysVet, un sistema web de código abierto para gestión veterinaria, usando la metodología ágil OSCRUM [5]. Comparte con BIOPET el interés en la reproducibilidad del artefacto (código abierto, proceso documentado), pero documenta el *proceso* de desarrollo ágil, no una evaluación empírica multimétrica del *producto* resultante.

3.2.5. **Putri, Hadi & Ramdani (2017) — Pruebas de rendimiento de un sistema web académico**

Caracteriza cuantitativamente el rendimiento de un sistema de admisión de estudiantes bajo carga, mediante *benchmarking* técnico contra el sistema real [24]. Es metodológicamente análogo a la evaluación de rendimiento de BIOPET con k6 (sección 8.3): ambos son estudios de *benchmarking* de un artefacto web real en un entorno académico, sin diseño experimental controlado. A diferencia de BIOPET, Putri et al. miden únicamente rendimiento, sin seguridad, usabilidad ni cobertura de pruebas.

3.2.6. **Yang et al. (2026) — Vulnerabilidades en implementaciones de JWT (NDSS)**

Auditoría sistemática de 43 implementaciones reales de JSON Web Token en diez lenguajes, con una herramienta de *fuzzing* dirigido (JWTeemo), que descubre 31 vulnerabilidades previamente desconocidas (20 con CVE asignado) [25]. BIOPET usa JWT como mecanismo central de autenticación (capítulo 9); este trabajo es la referencia académica más actual disponible sobre los riesgos reales de esa misma tecnología. A diferencia de Yang et al., que auditan *implementaciones/librerías* genéricas, BIOPET no reclama haber descubierto vulnerabilidades JWT como contribución científica: aporta evidencia de que una aplicación concreta que usa JWT fue auditada con múltiples técnicas y no presentó los patrones de mayor severidad detectables por ellas, sin afirmar estar “libre” de la categoría de vulnerabilidades de *fuzzing* dirigido a la librería subyacente (jjwt), que queda explícitamente fuera del alcance de la auditoría de BIOPET.

3.2.7. **Estdale & Georgiadou (2018) — Aplicación práctica de ISO/IEC 25010**

Discute la dificultad práctica de aplicar el modelo de calidad ISO/IEC 25010 a un producto de software real, más allá de la cita teórica de la norma [9]. Es paralelo metodológico directo al mapeo realizado en `docs/arquitectura/ISO-25010.md` (sección 6.7), donde se mapean las ocho características de ISO/IEC 25010:2011 contra evidencia real de BIOPET, declarando honestamente “no evaluada directamente” cuando no existe evidencia. A diferencia del trabajo de Estdale & Georgiadou, que discute el modelo en general, BIOPET aporta un mapeo concreto con rutas de evidencia verificables por cada subcaracterística.

3.2.8. Bucao, Quinones, Abong, Penuela & Sun (2023) — VETelgeuse

Sistema de gestión web y móvil para clínicas veterinarias pequeñas y medianas, evaluado con el cuestionario de usabilidad USE sobre 30 usuarios reales [26]. Confirma, junto con Cedeño Ochoa et al. y Llaneta et al., que el dominio de BIOPET (gestión veterinaria web) tiene evidencia académica real más allá de un único estudio. A diferencia de BIOPET, su validación empírica se limita a usabilidad, sin cobertura de pruebas, rendimiento bajo carga ni auditoría de seguridad.

3.2.9. Llaneta, Guelas, Labanan, Mercado & Sasis (2022) — TerraVet

Propone un *framework* web y móvil para conectar dueños de mascotas con clínicas veterinarias, publicado en ICIST 2022 (ACM), con una evaluación preliminar de la propuesta [27]. Es paralelo de dominio y de arquitectura web+móvil a BIOPET, pero no evalúa rendimiento bajo carga ni despliegue en la nube, ambos reportados en este informe (capítulos 8 y 10).

3.2.10. Zmaranda, Pop-Fele, Gyorödi, Gyorödi & Pecherle (2020) — CRUD con ORM vs. SQL directo

Compara el tiempo de ejecución de operaciones CRUD usando mapeadores objeto-relacionales (ORM) de .NET frente a acceso directo a datos [13]. Respaldar directamente la decisión de arquitectura de BIOPET de usar un **acceso híbrido**: Spring Data JPA para operaciones CRUD simples y SQL nativo en procedimientos almacenados para consultas complejas y validaciones cruzadas (sección 6.4). A diferencia de BIOPET, el estudio de Zmaranda et al. se limita a una sola pila (.NET) y no evalúa procedimientos almacenados ni agregaciones equivalentes a `fn_reporte_dashboard` o `fn_historial_clinico_mascota`.

3.3. Síntesis comparativa

Cuadro 3.1: Síntesis comparativa de los diez estudios incluidos frente a BIOPET.

Estudio	Dominio	Evaluación reportada	Diferencia principal frente a BIOPET
Pacheco et al. (2018)	Elicitación de requisitos	Madurez/efectividad comparada de técnicas	BIOPET no compara técnicas; aplica una y la traza formalmente
Bangor et al. (2008)	Usabilidad (SUS)	Benchmark de interpretación (68/80,3)	BIOPET usa el benchmark, no construye uno nuevo
Cedeño Ochoa et al. (2021)	Gestión veterinaria (Ecuador)	Mejora cualitativa de procesos	Sin evaluación multimétrica; BIOPET sí
Rodríguez et al. (2024) — OSCRUM	Gestión veterinaria, código abierto	Proceso ágil documentado	Documenta proceso, no evalúa empíricamente el producto
Putri et al. (2017)	Sistema web académico	Rendimiento bajo carga	Solo rendimiento; sin seguridad, usabilidad ni cobertura
Yang et al. (2026) — NDSS	Seguridad JWT (multi-lenguaje)	Vulnerabilidades descubiertas	Audita librerías genéricas, no una aplicación completa
Estdale & Georgiadou (2018)	Calidad de software (ISO/IEC 25010)	Discusión de evidenciabilidad	Guía general, no un mapeo concreto enlazado a evidencia
Bucao et al. (2023) — VETelgeuse	Gestión veterinaria (Filipinas)	Usabilidad (USE, n=30)	Solo usabilidad; sin rendimiento, seguridad ni cobertura
Llaneta et al. (2022) — TerraVet	Gestión veterinaria (marco web+móvil)	Propuesta de <i>framework</i> , evaluación preliminar	No evalúa rendimiento ni despliegue en la nube
Zmaranda et al. (2020)	Acceso a datos (ORM vs. SQL directo)	Tiempos de operaciones CRUD	Limitado a .NET; no cubre procedimientos almacenados ni agregaciones

Tomados individualmente, los diez estudios evalúan siempre **una sola dimensión a la vez**: elicitación de requisitos sin evaluar el producto resultante, un benchmark de usabilidad sin aplicarlo a un sistema concreto, el dominio veterinario sin evaluación multimétrica (tres veces: Cedeño Ochoa, Bucar, Llaneta), un proceso ágil sin evaluación empírica del producto, rendimiento sin seguridad ni usabilidad, seguridad JWT genérica sin evaluar una aplicación completa, el modelo de calidad discutido sin aplicarlo con evidencia enlazada a un caso real, o rendimiento de acceso a datos limitado a una sola pila tecnológica. BIOPET se posiciona frente a estos diez trabajos como una evaluación **multimétrica** de un único artefacto: cobertura de pruebas, rendimiento, usabilidad, calidad web, y seguridad por tres ángulos distintos (revisión manual OWASP, análisis dinámico ZAP, análisis estático SpotBugs/Find Security Bugs), todas enmarcadas bajo ISO/IEC 25010 como vocabulario común. Con la incorporación del bloque de Fred, el dominio veterinario exacto de BIOPET pasa de tener un solo estudio de contraste (Cedeño Ochoa et al.) a tres (sumando Bucar et al. y Llaneta et al.), aunque ninguno de los tres combina el contexto ecuatoriano con una evaluación multimétrica — esa combinación específica sigue sin evidencia externa directa. No se afirma que BIOPET sea metodológicamente superior a ninguno de estos trabajos: no existe una comparación empírica directa que lo demuestre; la diferencia señalada es de **amplitud de la evidencia reportada** sobre un mismo sistema, no de calidad relativa del artefacto.

3.4. Limitaciones de esta revisión

- Solo se incluyeron fuentes en español e inglés; podrían existir estudios relevantes en otros idiomas no cubiertos.
- El bloque de Jaime no documentó el conteo de candidatos evaluados y excluidos (solo el resultado final de 4 incluidos), a diferencia de los bloques de Zaida (15 candidatos, 12 exclusiones documentadas) y de Fred (13 candidatos por DOI, 2 descartados con motivo individual); este gap se señala explícitamente con línea punteada en el diagrama de flujo regenerado (figura 3.1), único bloque que aún no está completamente cuantificado.
- La selección no tuvo un segundo revisor independiente por estudio: cada bloque fue filtrado por una sola persona sobre su propio tema, sin validación cruzada entre integrantes — recurso real de un equipo de tres personas en un Proyecto Fin de Curso, declarado explícitamente en vez de omitirse. El bloque de Fred aplicó, además, un filtro temporal explícito (2020–2026) que los otros dos bloques no declararon como criterio formal — diferencia de criterio entre bloques declarada, no oculta.
- Con el bloque de Fred, el dominio veterinario exacto de BIOPET pasa de un estudio de contraste a tres, pero ninguno combina el contexto ecuatoriano con una evaluación multimétrica del sistema: esa combinación específica sigue subrepresentada en la literatura académica disponible, no por sesgo de búsqueda sino por escasez real, hallazgo reportado independientemente por los tres bloques de revisión.

Capítulo 4

Metodología

Este capítulo integra, en formato de informe académico, el borrador metodológico de Jaime (`docs/informe/borradores/jaime/metodologia-y-amenazas.md`), apoyado en el checklist ya completado `docs/checklists/ralph2021-engineering-research.md` [28].

4.1. Enfoque metodológico

BIOPET corresponde, de forma predominante, a una investigación de **ingeniería orientada al diseño y evaluación de un artefacto de software** (*Engineering Research*, también denominada *Design Science* en la literatura de ingeniería de software empírica). El equipo diseñó, construyó y evaluó empíricamente un sistema real — backend Spring Boot, frontend Angular e infraestructura Docker — frente a requisitos explícitos (`docs/requisitos/SRS.md`), usando múltiples fuentes de evidencia técnica: pruebas automatizadas y cobertura (JaCoCo), rendimiento (k6), usabilidad (SUS), calidad web automática (Lighthouse), y seguridad (OWASP, ZAP, SpotBugs/Find Security Bugs, auditoría de SQL dinámico).

Por qué *Engineering Research* y no una alternativa

- **No es un experimento controlado.** Ninguna medición manipula deliberadamente una variable independiente con asignación aleatoria a grupos de tratamiento/control. Las corridas de k6 miden el comportamiento del propio sistema bajo dos condiciones observacionales (caché fría y caliente), no un diseño experimental con grupos comparados.
- **No es un estudio de caso observacional tradicional.** BIOPET es un artefacto construido por el propio equipo dentro de un Proyecto Fin de Curso, no un fenómeno observado *in situ* en una clínica veterinaria real ya operando con el sistema.
- **Sí encaja con el patrón de *Engineering Research*,** cuyos atributos esenciales (descripción del artefacto, justificación de su relevancia, evaluación conceptual y empírica, discusión de limitaciones) están todos presentes y verificados en el checklist Ralph et al. ya completado.

Afirmaciones que este informe evita deliberadamente

No se afirma que BIOPET sea un experimento controlado (sin grupos de control ni asignación aleatoria); no se afirma validación industrial (toda la evaluación se realizó en un

entorno académico, sin usuarios ni profesionales de una organización externa real); no se afirma ninguna forma de certificación (ISO/IEC 25010 se usa como marco de clasificación de calidad, no como certificación obtenida); no se afirma causalidad experimental en ninguna medición; y no se afirma que los resultados sean universalmente generalizables.

4.2. Design Science Research: las seis actividades de Peffers et al.

Cuadro 4.1: Mapeo de BIOPET sobre las seis actividades de Peffers et al. [1].

Actividad	Estado	Evidencia
1. Identificación del problema y motivación	Completada	15 observaciones reales registradas y cerradas (docs/observaciones/OBSERVACIONES.md); SRS §1
2. Definición de objetivos de la solución	Completada	Requisitos REQ-F/REQ-NF verificables, trazados a HU/CU (docs/trazabilidad/matriz.csv)
3. Diseño y desarrollo	Completada	Modelo C4 (contexto/contenedores/componentes), 7 ADR, código fuente real en Backend/src/, frontend/src/
4. Demostración	Completada	make up de punta a punta; colecciones Postman; k6 contra el backend real; ZAP contra el contenedor real
5. Evaluación	Completada	JaCoCo, k6, SUS, Lighthouse, OWASP/ZAP/SpotBugs, todos con datos reales (sección 4.4)
6. Comunicación	Completada	Este informe reemplaza al de la Tercera Entrega, (este documento con cifras actualizadas de v1.0.0)

4.3. Goal–Question–Metric (GQM)

Goal: analizar el artefacto BIOPET (backend Spring Boot + frontend Angular + infraestructura Docker) **con el propósito de** evaluar **con respecto a** su calidad funcional, mantenibilidad, rendimiento, usabilidad, calidad web y seguridad, **desde el punto de vista** del equipo de desarrollo y de evaluadores académicos, **en el contexto de** un Proyecto Fin de Curso ejecutado en un entorno de desarrollo local con Docker y desplegado en producción sobre el plan gratuito de Render [3].

Cuadro 4.2: Preguntas y métricas GQM, con el valor de evidencia vigente al cierre de la Entrega Final.

Dimensión	Pregunta	Valor de evidencia (fuente)
Calidad funcional	¿Los requisitos declarados tienen respaldo trazable?	Matriz consistente frente al SRS (scripts/validate-traceability.sh)
Calidad funcional	¿La suite de pruebas pasa consistentemente?	205 pruebas, 0 fallos, 0 errores (Backend/target/surefire-reports/)
Mantenibilidad	¿Qué proporción del código se ejercita?	LINE 91,80 % / BRANCH 79,39 %, umbral 70 % (docs/mediciones/jacoco/METRICS.md)
Mantenibilidad	¿El proceso de build es reproducible y automatizado?	CI con 6 jobs verdes (.github/workflows/ci.yml)
Rendimiento	¿Latencia y error del endpoint cacheado bajo carga?	p95 6,4–21,1 ms (caliente/frío), error 0,0 % en 10 corridas (docs/mediciones/perf/REPORT.md)
Usabilidad	¿Qué tan usable perciben participantes reales el frontend?	SUS media 74,44/100, IC95 % [63,33, 85,56], n=18 (docs/mediciones/sus/REPORT.md)
Calidad web	¿El frontend cumple umbrales base de calidad web?	Performance/Accessibility/Best Practices/SEO cumplidos en las 12 corridas mobile+desktop del 2026-08-18 (docs/mediciones/lighthouse/)
Seguridad (dinámica)	¿Hay hallazgos de severidad alta por escaneo dinámico?	0 alertas altas, 1 informativa, ZAP Baseline (docs/mediciones/sec/zap/README.md)
Seguridad (estática)	¿El análisis estático detecta patrones de inyección SQL?	66 hallazgos totales, 0 de tipo SQL_* (docs/mediciones/sec/static-analysis/README.md)
Seguridad (SQL dinámico)	¿Los procedimientos almacenados construyen SQL de forma segura?	0 hallazgos, exit 0 (scripts/audit-sql-dynamic.sh)

4.4. Diseño de evaluación

Para cada fuente de evidencia empírica se documenta propósito, unidad evaluada, procedimiento reproducible, métrica principal y limitación conocida (detalle ampliado por dimensión en los capítulos 8 y 9):

1. **Pruebas automatizadas:** `mvn clean verify` (JUnit 5 + MockMvc + 2 pruebas de integración con Testcontainers contra PostgreSQL real). Métrica: pruebas/fallos/errores.
2. **Cobertura JaCoCo:** regla BUNDLE sobre todo el módulo backend; contadores LINE/BRANCH/COMPLEXITY.
3. **Rendimiento (k6):** 5 corridas en frío + 5 en caliente contra `GET /api/mascotas` sobre HTTPS/TLS 1.3; IC 95 % con distribución t de Student; prueba de Wilcoxon pareada entre condiciones fría/caliente.
4. **Usabilidad (SUS):** cuestionario de Brooke (1996) sin modificar, aplicado a 18 participantes externos al equipo, con consentimiento informado previo.
5. **Calidad web (Lighthouse):** `npx @lhci/cli autorun` contra el frontend servido por contenedor real, perfiles móvil simulado y escritorio, 3 corridas por ruta y por perfil (12 corridas en la re-ejecución final del 2026-08-18).
6. **Seguridad dinámica (ZAP Baseline):** escaneo no autenticado contra el backend real dentro de la red Docker Compose.
7. **Seguridad estática (SpotBugs + Find Security Bugs):** análisis de bytecode compilado, `effort=Max`, `threshold=Low`.
8. **Auditoría de SQL dinámico:** análisis de patrones inseguros (`EXECUTE`, concatenación) sobre `db/procs/*.sql`.
9. **Trazabilidad de requisitos:** validación cruzada automática SRS ↔ matriz.

Cada método declara su propia limitación en el capítulo correspondiente y en el capítulo 14; ninguna cifra de este informe se presentó sin una fuente citada.

4.5. Reproducibilidad

El sistema completo se reconstruye desde una clonación limpia con un único comando (`make up`), con imágenes de terceros fijadas por *digest sha256* para evitar derivas silenciosas entre reconstrucciones. Los scripts que generan cada pieza de evidencia (`scripts/archive-jacoco-evidence.sh`, `scripts/run-zap-baseline.sh`, `scripts/audit-sql-dynamic.sh`, `scripts/validate-traceability.sh`, `scripts/perf-analysis.py`, `scripts/analisis-sus.py`, `scripts/run-lighthouse.sh`, entre otros) están versionados, de modo que la evidencia no es solo consultable sino regenerable. El detalle completo de reproducibilidad de despliegue se documenta en el capítulo 10.

4.6. Amenazas a la validez (resumen metodológico)

Las amenazas a la validez de cada medición se documentan en detalle en el capítulo 14, clasificadas según las cuatro categorías estándar (interna, externa, de constructo y de conclusión). Este capítulo de metodología se limita a declarar el principio que las gobierna: cada amenaza documentada incluye su efecto posible, la mitigación ya aplicada (si existe) y el riesgo residual que persiste, sin minimizar ninguna de ellas.

Capítulo 5

Ingeniería de requisitos

Este capítulo integra `docs/requisitos/SRS.md` (Especificación de Requisitos de Software, con patrón ISO/IEC/IEEE 29148), las historias de usuario, los casos de uso y `docs/requisitos/CHANGELOG-REQ.md`. El SRS distingue explícitamente requisitos **verificados**, **implementados** (sin prueba automatizada dedicada) y **pendientes**; este capítulo respeta esa misma distinción: ningún requisito que el SRS marca como pendiente se declara aquí como cumplido.

5.1. Priorización MoSCoW

Los requisitos se priorizan con la técnica MoSCoW (Must, Should, Could, Won't). El núcleo *Must Have* corresponde al compromiso funcional operativo de esta entrega: autenticación, autorización por rol y por propiedad, y CRUD de mascotas. Los módulos de citas, consultas y vacunas, incorporados durante la Unidad IV, se formalizaron como REQ-F-023 a REQ-F-025 (sección 5.2.1).

5.2. Requisitos funcionales implementados y verificados

Los requisitos REQ-F-001 a REQ-F-012 y REQ-F-021 cubren registro de usuario, login/logout/refresh por cookies, autorización por rol, perfil del usuario autenticado, y el CRUD completo de mascotas con resumen por especie; todos con estado **verificado** en el SRS, respaldados por pruebas automatizadas de controlador/servicio (capítulo 8).

5.2.1. Requisitos incorporados en la Unidad IV (REQ-F-023 a REQ-F-025)

Estos tres requisitos formalizan funcionalidades reales incorporadas al backend durante la Unidad IV, distintas de Citas y Consultas (que ya tenían requisito formal histórico propio — REQ-F-015 y REQ-F-013 respectivamente — y no se duplican aquí):

- **REQ-F-023 — Gestión administrativa de usuarios** (`UsuarioController/UsuarioService`): un ADMIN lista, consulta, crea, actualiza y da de baja lógica cuentas de cualquier rol, sin poder autoescalar el rol de su propia cuenta. Estado: **verificado** (`UsuarioControllerTest`, 16 pruebas).

- **REQ-F-024 — Gestión de vacunas** (`VacunaController`): registro, consulta (global, por mascota o por identificador), actualización y baja lógica de vacunas, con propiedad restringida para `ROLE_DUENO`. Estado: **verificado** (`VacunaControllerTest`, 10 pruebas; colección Postman `BIOPET-Vacunas.postman_collection.json`); la operación `GET /api/vacunas/mascota/{mascotaId}` no tiene prueba automatizada dedicada.
- **REQ-F-025 — Consulta de información externa de especies** (`ExternalApiController`): consulta de datos de una especie (nombre científico, hábitat, dieta) desde un servicio externo (API Ninjas), con caché Redis *cache-aside* para evitar llamadas repetidas. Estado: **implementado** (sin prueba automatizada dedicada; no se marca “verificado” siguiendo la misma convención ya usada en el resto del SRS); evidencia empírica manual de la comparación caché fría/caliente en `docs/u4/evidencias/fred/redis-cache-comparacion.md`.

Rendimiento bajo carga de citas, consultas y vacunas no fue medido específicamente con k6 (la medición de rendimiento del capítulo 8, sección 8.3, se limita al endpoint de mascotas), limitación declarada en el capítulo 14.

Las seis rutinas almacenadas de PostgreSQL (`fn_resumen_mascotas_por_especie`, `fn_historial_clinico_mascota`, `fn_reporte_dashboard`, `fn_siguiente_numero_ficha`, `sp_actualizar_estado_citas_masivas`, `sp_registrar_consulta_validada`) respaldan estos módulos con acceso JPA formal (sección 6.4).

5.3. Requisitos pendientes, heredados de la Entrega 1A

Los requisitos REQ-F-013 a REQ-F-020 y REQ-F-022 permanecen, con honestidad, en estado **pendiente** o **parcial** en el SRS, sin implementación de backend ni de frontend a la fecha de este informe salvo lo explicitado:

Cuadro 5.1: Requisitos funcionales pendientes o parciales heredados de la Entrega 1A (fuente: `docs/requisitos/SRS.md`).

Requisito	Descripción	Prioridad	Estado
REQ-F-013	Registro/consulta de historial clínico cronológico como módulo independiente	Should	Pendiente
REQ-F-014	Registro de medicamentos prescritos	Could	Pendiente
REQ-F-015	Calendario interactivo de citas (módulo básico de citas ya existe; el calendario interactivo no)	Should	Pendiente
REQ-F-016	API de recepción de telemetría IoT	Could	Pendiente
REQ-F-017	Recomendaciones clínicas generadas por IA externa	Could	Pendiente
REQ-F-018	Facturación digital y registro de pagos	Should	Pendiente
REQ-F-019	Reportes estadísticos exportables (PDF/Excel)	Could	Pendiente
REQ-F-020	Auditoría de operaciones del sistema	Should	Parcial
REQ-F-022	Notificaciones por correo electrónico	Could	Pendiente

Nota de honestidad sobre REQ-F-015 y REQ-F-013. El backend expone `CitaController` y `ConsultaController` reales, con endpoints funcionales y respaldados por las rutinas almacenadas de la Unidad IV (sección 5.2.1); sin embargo, el SRS mantiene REQ-F-013 (historial clínico cronológico completo, con consulta longitudinal) y la parte de *calendario interactivo* de REQ-F-015 como pendientes, porque esas funcionalidades específicas — más allá del CRUD básico de consultas y citas ya implementado — no están completas ni tienen frontend dedicado. Este informe reporta ambos estados tal como los declara el SRS vigente, sin resolver la aparente tensión ampliando el alcance por cuenta propia: se trata como lo que es, una actualización parcial del SRS que aún no reclasificó explícitamente cada subrequisito heredado frente al código nuevo de la Unidad IV, y se documenta como limitación de consistencia documental en el capítulo 14.

REQ-F-020 (auditoría) se marca **parcial**, no **pendiente**: el registro de eventos de autenticación (`AUTH_AUDIT`, capítulo 9) sí existe y está verificado, pero no cubre auditoría genérica de todas las operaciones CRUD sobre mascotas ni de los módulos de citas/consultas/vacunas.

5.4. Requisitos no funcionales relevantes

Cuadro 5.2: Requisitos no funcionales relevantes citados en este informe (fuente: docs/requisitos/SRS.md, sección 3.2).

Requisito	Descripción	Estado
REQ-NF-001	Rendimiento del listado de mascotas (≤ 200 ms p95 caliente, ≤ 500 ms fría)	Verificado
REQ-NF-002	Comunicación cifrada obligatoria (HTTPS/TLS)	Verificado (TLS de desarrollo)
REQ-NF-003	Expiración configurable de tokens JWT	Verificado
REQ-NF-004	Claims estándar del JWT (RFC 7519)	Verificado
REQ-NF-005	Interfaz responsiva (320–1440px) / Lighthouse	Verificado (sección 8.5)
REQ-NF-006	Compatibilidad de navegadores (Chrome/Firefox/Edge)	Pendiente de evidencia archivada
REQ-NF-008	Cifrado de contraseñas (BCrypt)	Verificado
REQ-NF-009	Registro de eventos de autenticación (OWASP A09)	Verificado
REQ-NF-010	Limitación de intentos de login (OWASP A07)	Verificado
REQ-NF-011	Arquitectura en capas (controller/service/repository)	Verificado
REQ-NF-012	Caché del listado de mascotas con TTL externo	Verificado parcialmente (TTL confirmado; sin hit ratio medido)
REQ-NF-013	Estrategia híbrida de acceso a datos (JPA + procedimientos almacenados)	Verificado (capítulo 6)

El requisito de interfaz responsiva/calidad web (REQ-NF-005) está **verificado**: las cuatro categorías de Lighthouse (Performance, Accessibility, Best Practices, SEO) cumplen su umbral en la corrida del 2026-08-18, en ambos perfiles mobile y desktop (sección 8.5) — superando el estado “verificado parcialmente” que declaraba una revisión anterior del SRS, fechada antes de esa corrida. REQ-NF-012 (hit ratio de caché Redis) permanece verificado solo parcialmente: existe evidencia de TTL y de existencia de clave bajo carga, pero no una medición explícita de `keyspace_hits/keyspace_misses` a lo largo del tiempo. No existe, en el SRS actual, un requisito no funcional formal dedicado específicamente a la usabilidad medida con SUS: la medición SUS (sección 8.4) responde a un criterio de evaluación de la Guía de la Entrega Final, no a un REQ-NF numerado del SRS.

5.5. Trazabilidad y estado de la validación automática

`scripts/validate-traceability.sh` valida de forma cruzada que todo requisito del SRS tenga fila en `docs/trazabilidad/matriz.csv` con historia de usuario y caso de uso asociados. El detalle completo de la matriz se presenta en el capítulo 11.

5.6. Marco INCOSE

La redacción de cada requisito sigue el patrón “El sistema deberá...” con entradas, resultado esperado y criterios de aceptación verificables, alineado con las características de calidad de requisitos de la *INCOSE Guide to Writing Requirements* [6] (`docs/checklists/incose2023-req.md`): en particular *Unambiguous* y *Verifiable*, aplicadas explícitamente al cerrar la ambigüedad histórica de REQ-F-017 (observación OBS-04, ya cerrada).

5.7. Cambios entre versiones

`docs/requisitos/CHANGELOG-REQ.md` documenta la evolución del SRS entre la Entrega 1A, la Entrega 1B, la Tercera Entrega y la Entrega Final, incluyendo la incorporación de REQ-F-022 (recuperación formal de RF-07, cierre de OBS-02) y de REQ-F-023 a REQ-F-025 (Unidad IV). Ningún identificador de requisito se reutilizó para dos funcionalidades distintas.

Capítulo 6

Diseño y arquitectura

Este capítulo integra la documentación arquitectónica versionada en `docs/diagrams/` (modelo C4 [8]) y las decisiones de arquitectura relevantes (`docs/adr/`), actualizada para reflejar el estado v1.0.0 (PostgreSQL 18/Valkey 8 en producción, acceso JPA formal a procedimientos almacenados).

6.1. Arquitectura general

BIOPET sigue una arquitectura de tres niveles: un *frontend* Angular 17 (aplicación de página única, componentes *standalone*), un *backend* Spring Boot 3.2 sobre Java 21 que expone una API REST [29], una base de datos relacional PostgreSQL [30] (16 en desarrollo local, 18 en producción) gestionada con Flyway (migraciones V1 a V6), y Redis/Valkey [31] (7 en desarrollo, Valkey 8 en producción) usado tanto para la lista negra de tokens JWT revocados como para la caché de lecturas de mascotas. Todo el sistema se levanta mediante Docker Compose y un `Makefile` con un único comando reproducible (`make up`).

6.2. C4 Nivel 2 — Contenedores

El diagrama de contenedores (`docs/diagrams/c4-contenedores/`) describe los cuatro contenedores del sistema: el frontend Angular, el backend Spring Boot, PostgreSQL y Redis/Valkey, junto con los protocolos reales entre ellos (HTTPS/JSON con cookies entre frontend y backend; JDBC entre backend y PostgreSQL; el cliente Redis de Spring Data entre backend y Redis/Valkey).

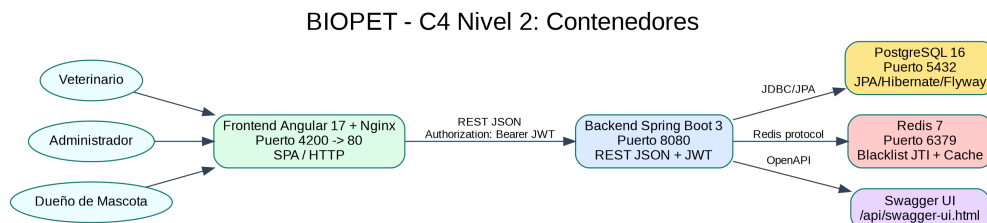


Figura 6.1: Diagrama C4 Nivel 2 (contenedores) de BIOPET: frontend, backend, PostgreSQL y Redis/Valkey, con los protocolos reales entre ellos.

6.3. C4 Nivel 3 — Componentes del backend

El diagrama de componentes del backend fue elaborado mediante inspección directa del código fuente (constructores, llamadas directas, configuración de Spring), no por convención de nombres. A los componentes ya documentados en la Tercera Entrega (controladores de autenticación, mascotas y usuario; servicios de seguridad; manejo de errores; persistencia; infraestructura del servidor) se suman, en v1.0.0, los controladores de la Unidad IV: **CitaController**, **ConsultaController**, **VacunaController** y **ExternalApiController**, junto con el repositorio formal de acceso a procedimientos almacenados.

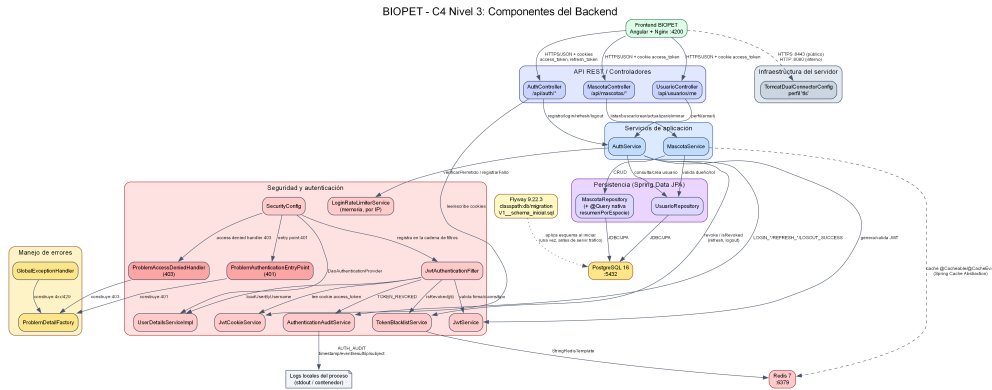


Figura 6.2: Diagrama C4 Nivel 3 (componentes) del backend de BIOPET.

6.4. Acceso JPA formal a procedimientos almacenados

Las seis rutinas de `db/procs/` son objetos PostgreSQL de tipo `PROCEDURE`, invocadas desde el backend mediante `@Procedure/@NamedStoredProcedureQuery` de Spring Data JPA, en vez de `@Query(nativeQuery=true)` plano (usado en la Tercera Entrega solo para la primera rutina). Las cuatro rutinas con prefijo `fn_*` originalmente eran `FUNCTION`; se reclasificaron a `PROCEDURE` (conservando el nombre por compatibilidad) para permitir esa invocación formal.

Cuadro 6.1: Catálogo de las seis rutinas almacenadas de BIOPET (fuente: docs/basedatos/CATALOGOSP.md).

Rutina	Categoría	Propósito
fn_resumen_- mascotas_por_- especie	Agregado	Resumen de mascotas activas por especie, filtrable por dueño
fn_historial_- clinico_mascota	Multi-tabla	Historial clínico consolidado: datos, dueño, conteos de consultas/citas, última consulta, última y próxima vacuna
fn_reporte_- dashboard	Reporte	Indicadores de dashboard para un rango de fechas
fn_siguiente_- numero_ficha	Código secuencial	Siguiente número de ficha PREFIJO-NNNNNN
sp_actualizar_- estado_citas_- masivas	Actualización masiva	UPDATE controlado del estado de citas por veterinario/fecha límite
sp_registrar_- consulta_validada	Validación cruzada	Registra una consulta solo si la mascota y el veterinario son válidos y activos

Estrategia de migración sin romper compatibilidad. La reclasificación no se aplicó reescribiendo la migración Flyway original (V5__procedimientos_biopet.sql, que pudo haberse aplicado ya en bases persistentes: reescribirla habría producido `checksum mismatch`). En su lugar, una migración nueva y aditiva (V6__formalizar_procedimientos_jpa.sql) hace `DROP FUNCTION IF EXISTS` de las cuatro rutinas creadas por V5 y las vuelve a crear como `PROCEDURE` con su firma final. Flyway aplica V1 a V6 en orden sobre un checkout nuevo, y solo V6 sobre una base que ya tenga V1-V5 aplicadas.

```

PS C:\Proyectos\PFC--VET-ENTRIB\Backend> make down
>> make up
>> docker compose ps
make: *** No rule to make target 'down'. Stop.
make: *** No rule to make target 'up'. Stop.
NAME                CREATED              IMAGE                STATUS                PORTS                COMMAND                SERVICE
biopet-backend      2 minutes ago       pfc-vet-entrib-backend Up 2 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp "java -jar /app/app..." backend
biopet-frontend      2 minutes ago       pfc-vet-entrib-frontend Up About a minute (unhealthy) 0.0.0.0:4200->80/tcp, [::]:4200->80/tcp "/docker-entrypoint..." frontend
biopet-postgres      2 minutes ago       postgres:16-alpine@sha256:57c72fd2a128e416c7fcc49958864df5301e940bca0a56f58fddf30ffcc07777 "docker-entrypoint.s..." postgres
biopet-redis         3 hours ago         redis:7-alpine@sha256:e7723ff73d963f5cc6d9c4643ea3d989527a402a319239054e9472a7fb9219a2 "docker-entrypoint.s..." redis
PS C:\Proyectos\PFC--VET-ENTRIB\Backend> $response = Invoke-WebRequest -Uri "http://localhost:8080/api/auth/login" -Method POST -ContentType "application/json" -Body '{"email":"admin@biopet.ec","password":"Admin123"}' -UseBasicParsing
>> $token = ($response.Content | ConvertFrom-Json).accessToken
>>
>> Invoke-WebRequest -Uri "http://localhost:8080/api/mascotas/resumen-especies" -Method GET -Headers @{Authorization="Bearer $token"} -UseBasic
Parsing
StatusCode      : 200
StatusDescription :
Content         : [{"especie":"Perro","total":1}]
RawContent      : HTTP/1.1 200
Vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Pragma: no-cache
X-Frame-Options: SAMEORIGIN
Content-S...
Forms           :
Headers         : {[Vary, Origin,Access-Control-Request-Method,Access-Control-Request-Headers], [X-Content-Type-Options, nosniff], [X-XSS-Protection, 0], [Pragma, no-cache]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      :
    
```

Figura 6.3: Acceso híbrido a procedimientos almacenados: invocación formal JPA sobre las rutinas reclasificadas como `PROCEDURE`.

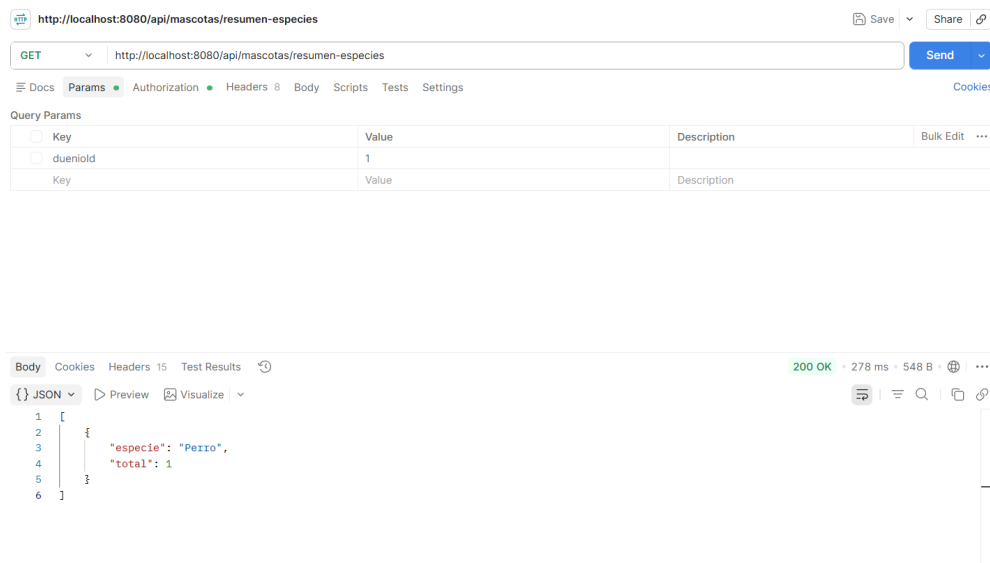


Figura 6.4: Invocación real de una rutina almacenada verificada desde Postman.

6.5. Decisiones de arquitectura relevantes (ADR)

Cuadro 6.2: ADR vigentes del repositorio.

ADR	Decisión	Estado
ADR-002	Migración de la pila de backend de ASP.NET Core 8 a Java 21 / Spring Boot 3.2.	Aceptado
ADR-003	Revocación de JWT mediante lista negra en Redis, indexada por <code>jti</code> .	Aceptado
ADR-004	Estrategia de base de datos reproducible: Flyway como fuente de verdad del esquema, más scripts de inicialización de Docker con privilegios mínimos.	Aceptado
ADR-005	Reproducibilidad del despliegue: <code>Makefile</code> y pinning de imágenes por digest <code>sha256</code> .	Aceptado
ADR-006	Decisiones de autenticación y seguridad: JWT por cookies, claims, revocación, rate limiting, cabeceras, TLS.	Aceptado
ADR-007	Acceso formal JPA a procedimientos almacenados de PostgreSQL (<code>@Procedure/@NamedStoredProcedureQuery</code>), reclasificación <code>fn_*</code> a <code>PROCEDURE</code> .	Aceptado

ADR-001 (selección inicial de ASP.NET Core 8 en la Entrega 1A) quedó formalmente superado por ADR-002 y se conserva únicamente como registro histórico.

6.6. Patrones y decisiones de seguridad relevantes al diseño

El patrón *cache-aside* declarativo de Spring (@Cacheable/@CacheEvict en MascotaService) respalda las lecturas de GET /api/mascotas, con Redis/Valkey como almacén de caché. La autenticación usa JWT firmado con HMAC-SHA256, transportado en cookies HttpOnly+Secure+SameSite=Strict, con revocación real vía lista negra en Redis y rate limiting de login en memoria (detalle completo en el capítulo 9). El formato uniforme de error sigue RFC 7807 (ProblemDetail) [11].

6.7. ISO/IEC 25010: marco de calidad

docs/arquitectura/ISO-25010.md mapea las ocho características de calidad de ISO/IEC 25010:2011 (Funcionalidad, Eficiencia de desempeño, Compatibilidad, Usabilidad, Confiabilidad, Seguridad, Mantenibilidad, Portabilidad) contra evidencia real de BIOPET, siguiendo el enfoque de aplicación práctica de Estdale & Georgiadou [9] (sección 3). Cada subcaracterística se declara **evaluada** (con ruta de evidencia citada) o **no evaluada directamente** cuando no existe medición específica — por ejemplo, *Interoperability* más allá de la API REST documentada con OpenAPI, o *Adaptability* más allá de la configuración externalizada por variables de entorno.

Cuadro 6.3: Mapeo resumido de ISO/IEC 25010 sobre evidencia de BIOPET.

Característica	Evidencia principal
Funcionalidad (<i>Functional Suitability</i>)	Matriz de trazabilidad, SRS, 205 pruebas automatizadas
Eficiencia de desempeño	10 corridas k6 (docs/mediciones/perf/REPORT.md)
Compatibilidad	Docker Compose multi-servicio, API REST documentada con OpenAPI
Usabilidad	SUS (n=18), atributos ARIA verificados en código
Confiabilidad	0,0 % tasa de error en las 10 corridas k6, healthcheck de producción UP
Seguridad	OWASP Top 10 manual, ZAP Baseline, SpotBugs/Find Security Bugs (capítulo 9)
Mantenibilidad	Cobertura JaCoCo 91,80 %/79,39 %, CI con 6 jobs
Portabilidad	Imágenes Docker por digest sha256, despliegue reproducible en Render

Capítulo 7

Implementación

Este capítulo describe, a nivel de diseño y sin degenerar en un listado de código fuente, las implementaciones principales de BIOPET v1.0.0.

7.1. Autenticación y autorización

El backend emite JWT firmados con HMAC-SHA256 (jjwt 0.12.6), transportados en cookies `access_token/refresh_token` (`HttpOnly+Secure+SameSite=Strict`). Cada token contiene diez *claims*: los siete estándar de RFC 7519 [10] (`iss`, `sub`, `aud`, `iat`, `nbfi`, `exp`, `jti`) y tres personalizados (`email`, `rol`, `typ`). La autorización combina rol (`@PreAuthorize`) y propiedad (verificación explícita de que un `ROLE_DUENO` solo accede a sus propias mascotas). El logout es idempotente y revoca el token en Redis mediante su `jti`. Detalle completo en el capítulo 9.

7.2. Gestión de usuarios

`UsuarioController` expone el perfil del usuario autenticado (`GET /api/usuarios/me`). El registro (`POST /api/auth/registro`) crea siempre un usuario con `ROLE_DUENO`; los demás roles (`ADMIN`, `VETERINARIO`, `AUXILIAR`) se asignan mediante el usuario administrador sembrado o por un administrador ya autenticado, no de forma autoasignada por el registro público.

7.3. Gestión de mascotas

CRUD completo (`GET/POST/PUT/DELETE` sobre `/api/mascotas` y `/api/mascotas/{id}`), con eliminación lógica (`activo=false`), no física, y un endpoint de resumen por especie respaldado por `fn_resumen_mascotas_por_especie`.

7.4. Citas

`CitaController` gestiona la programación de citas veterinarias (alta, consulta, cambio de estado). La actualización masiva del estado de citas vencidas de un veterinario se resuelve con la rutina `sp_actualizar_estado_citas_masivas` (transacción única en PostgreSQL, no un bucle de actualizaciones individuales desde el backend).

7.5. Consultas

`ConsultaController` registra atenciones veterinarias. El registro usa la rutina `sp_registrar_consulta_validada`, que valida en la propia base de datos — antes de insertar — que la mascota exista y esté activa, y que el veterinario referenciado tenga rol `VETERINARIO` o `ADMIN` y esté activo, lanzando `RAISE EXCEPTION` si no, en vez de delegar esa validación únicamente a la capa de servicio de Java.

7.6. Vacunas

`VacunaController` registra y consulta vacunas aplicadas a una mascota, insumo de `fn_historial_clinico_mascota` para mostrar la última y la próxima vacuna de una mascota en el resumen consolidado.

7.7. Integración con API externa

`ExternalApiController` consume un servicio externo desde el backend. Se documenta como **implementado**, no **verificado**: no cuenta con una prueba automatizada dedicada a la fecha de este informe (sección 5.2.1).

7.8. Caché (patrón cache-aside)

`MascotaService` usa la abstracción de caché de Spring (`@Cacheable` para lecturas, `@CacheEvict` para invalidación tras escritura) contra Redis/Valkey. El TTL de caché se externaliza por variable de entorno, no está codificado en el fuente. El listado 7.1 muestra el fragmento real de `MascotaService` que implementa este patrón: el listado paginado se cachea con una clave compuesta (correo, página, tamaño, orden) para que cada combinación de parámetros tenga su propia entrada, y toda operación de escritura invalida el caché completo del listado.

```

1  @Cacheable(value = "mascotas",
2      key = "#email + '-' + #pageable.pageNumber + '-' +
3          #pageable.pageSize + '-' + #pageable.sort.toString()")
4  @Transactional(readOnly = true)
5  public Page<MascotaResponse> listar(Pageable pageable, String email) {
6      Usuario usuario = usuarioRepository
7          .findByEmailAndActivoTrue(email)
8          .orElseThrow(() -> new RecursoNoEncontradoException(
9              "Usuario no encontrado"));
10
11      if (usuario.getRol() == Rol.ROLE_DUEÑO) {
12          return mascotaRepository
13              .findAllByDuenioIdAndActivoTrue(usuario.getId(), pageable)
14              .map(this::toResponse);
15      }
16      return mascotaRepository.findAllByActivoTrue(pageable)
17          .map(this::toResponse);
18  }
19
20  @CacheEvict(value = "mascotas", allEntries = true)
21  @Transactional
    
```

```

22 public MascotaResponse crear(MascotaRequest request) {
23     // ... validacion y persistencia (omitida por brevedad)
24 }
    
```

Listing 7.1: Patrón *cache-aside* declarativo en `MascotaService.java` (fragmento real).

7.9. Procedimientos almacenados

Ver capítulo 6, sección 6.4, para el catálogo completo y la estrategia de migración que preserva compatibilidad con bases ya migradas. El listado 7.2 muestra el repositorio dedicado a la invocación formal de las seis rutinas almacenadas: cada método usa `@Procedure`, sin ningún CRUD genérico expuesto (el repositorio extiende `Repository`, no `JpaRepository`, deliberadamente).

```

1 public interface ProcedimientoBiopetRepository
2     extends Repository<Mascota, Long> {
3
4     @Procedure(name = "fn_resumen_mascotas_por_especie")
5     List<ResumenEspecie> resumenPorEspecie(Long dueñoId);
6
7     @Procedure(procedureName = "sp_registrar_consulta_validada",
8         outputParameterName = "p_consulta_id")
9     Long registrarConsultaValidada(
10         @Param("p_mascota_id") Long mascotaId,
11         @Param("p_veterinario_id") Long veterinarioId,
12         @Param("p_motivo") String motivo,
13         @Param("p_diagnostico") String diagnostico,
14         @Param("p_tratamiento") String tratamiento,
15         @Param("p_observaciones") String observaciones);
16 }
    
```

Listing 7.2: Acceso JPA formal a procedimientos almacenados en `ProcedimientoBiopetRepository.java` (fragmento real).

7.10. Migraciones Flyway

El esquema se gestiona **exclusivamente** con Flyway como única fuente de verdad, aplicando en orden `V1__schema_inicial.sql`, `V2__citas.sql`, `V3__consultas.sql`, `V4__vacunas.sql`, `V5__procedimientos_biopet.sql` y `V6__formalizar_procedimientos_jpa.sql`, verificado desde un volumen de PostgreSQL completamente vacío. La arquitectura de arranque reproducible en Docker es, en su versión final v1.0.0:

1. PostgreSQL inicia desde un volumen vacío.
2. `docker-entrypoint-initdb.d` ejecuta únicamente `db/roles-bootstrap.sql`: un *bootstrap* mínimo que crea el rol `biopet_app` **sin ninguna dependencia de objetos de esquema** (sin tablas, secuencias, funciones ni procedimientos), porque `application.yml` configura ese rol como el usuario del *pool* principal de Spring (Hikari/JPA) y debe existir antes de que el contenedor del backend intente conectarse, independientemente del estado de Flyway.
3. Flyway ejecuta realmente las migraciones **V1 a V6** en orden, sobre el esquema todavía vacío (sin `baseline-on-migrate` enmascarando un esquema preexistente).

4. `afterMigrate.sql` (*callback* de Flyway) otorga a `biopet_app` los privilegios sobre los objetos de esquema recién creados por V1-V6, una vez que esos objetos ya existen.
5. `DataInitializer` (`CommandLineRunner seedAdmin`) siembra el usuario administrador de forma idempotente **después** de que Flyway termina, dentro del propio ciclo de arranque de Spring Boot, no como script de Postgres.
6. `db/schema.sql`, `db/seed.sql`, `db/roles.sql` y `db/procs/` **no se ejecutan automáticamente** en este flujo: se conservan en el repositorio como catálogo, evidencia histórica y documentación de referencia (`docs/basedatos/CATALOGOSP.md`), no como parte activa del *bootstrap* de Docker.

Esta arquitectura corrige un diseño anterior en el que `db/schema.sql` duplicaba V1 y, montado en `docker-entrypoint-initdb.d` junto con `spring.flyway.baseline-on-migrate`, enmascaraba la migración real detrás de un *baseline* sobre un volumen que aparentaba ser nuevo. La versión final v1.0.0 elimina esa duplicidad: Flyway es la única vía por la que se crea el esquema, verificable ejecutando `make up` desde una clonación limpia y observando V1 a V6 aplicarse en orden en los logs del backend.

7.11. Manejo uniforme de errores (RFC 7807)

Todos los errores de la API usan el formato `application/problem+json` (RFC 7807 [11]), con `type`, `title`, `status`, `detail` e `instance` (`GlobalExceptionHandler`, `ProblemDetailFactory`). El `detail` nunca incluye *stack trace* ni el valor bruto de un parámetro inválido. El listado 7.3 muestra la fábrica real que construye cada `ProblemDetail`, reutilizada por todos los manejadores de excepción del backend.

```

1 public static ProblemDetail build(HttpStatus status, ProblemType type,
2     String title, String detail, HttpServletRequest request) {
3     ProblemDetail problemDetail =
4         ProblemDetail.forStatusAndDetail(status, detail);
5     problemDetail.setType(type.uri());
6     problemDetail.setTitle(title);
7     problemDetail.setInstance(URI.create(request.getRequestURI()));
8     return problemDetail;
9 }
```

Listing 7.3: Fábrica de errores RFC 7807 en `ProblemDetailFactory.java` (fragmento real).

7.12. Frontend

Aplicación Angular 17.3 con componentes *standalone* y formularios reactivos. `core/auth.service.ts/auth.guard.ts` gestionan la sesión por cookies (`withCredentials: true`, sin JWT en `localStorage`); `core/http-error.interceptor.ts` traduce el `ProblemDetail` del backend a mensajes legibles y reintenta `refresh` ante un 401 en rutas protegidas; `features/login.component.ts` y `features/mascotas.component.ts` usan atributos ARIA (`aria-invalid`, `aria-describedby`, `role="alert"`, `aria-live`) para anunciar errores a lectores de pantalla, en línea con WCAG 2.1 [32].

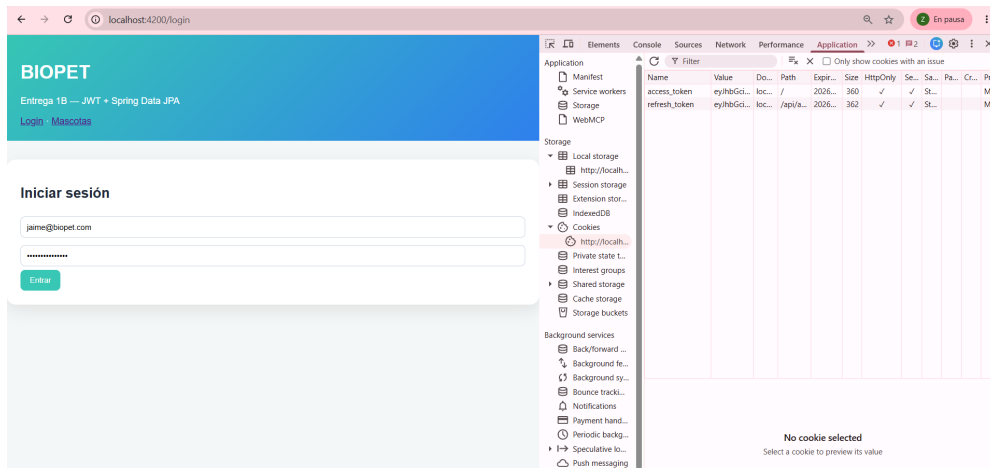


Figura 7.1: Pantalla de inicio de sesión del frontend de BIOPET.

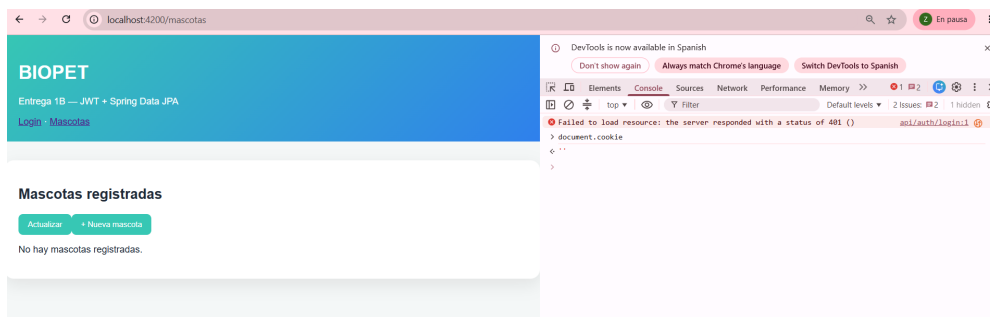


Figura 7.2: CRUD de mascotas en el frontend: listado, alta y edición.

7.13. Persistencia

Spring Data JPA sobre PostgreSQL, con consultas derivadas para `UsuarioRepository` y `MascotaRepository`, y acceso formal (`@Procedure`/`@NamedStoredProcedureQuery`) para las seis rutinas almacenadas descritas en el capítulo 6. Hibernate/JPA se conecta en tiempo de ejecución con la cuenta de privilegios mínimos `biopet_app`; Flyway usa la cuenta propietaria `biopet_user`, separando la conexión que aplica migraciones de la que sirve tráfico de la aplicación.

Capítulo 8

Pruebas y calidad

8.1. Pruebas automatizadas del backend

La suite completa del backend, ejecutada con `mvn clean verify` (verificado directamente contra `Backend/target/surefire-reports/` en esta fase de cierre), reporta:

Cuadro 8.1: Resultado real de la suite de pruebas del backend.

Métrica	Valor
Pruebas ejecutadas	205
Fallos	0
Errores	0
Omitidas	0
Resultado del build	BUILD SUCCESS

```
[INFO] Results:
[INFO] Tests run: 108, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.12:report (report) @ biopet-backend ---
[INFO] Loading execution data file C:\Users\jaime\Desktop\TerceraEntrega\PFC-VET-ENTR3-v0.9.0-rc\B
ackend\target\jacoco.exec
[INFO] Analyzed bundle 'BIOPET Backend' with 26 classes
[INFO] --- jar:3.3.0:jar (default-jar) @ biopet-backend ---
[INFO] Building jar: C:\Users\jaime\Desktop\TerceraEntrega\PFC-VET-ENTR3-v0.9.0-rc\Backend\target\
```

Figura 8.1: Salida real de `mvn clean verify`: pruebas del backend con BUILD SUCCESS.

8.2. Cobertura JaCoCo

JaCoCo [33] mide y verifica automáticamente la cobertura con una regla BUNDLE sobre todo el módulo backend. El umbral automático configurado en `Backend/pom.xml` se elevó de 60 % (Tercera Entrega) a 70 % para v1.0.0.

Cuadro 8.2: Cobertura medida por `jacoco:check` (fuente: `docs/mediciones/jacoco/METRICS.md`).

Contador	Cubierto	No cubierto	Cobertura	Umbral
LINE	885	79	91,80 %	70 %
BRANCH	181	47	79,39 %	70 %
COMPLEXITY	299	77	79,52 %	60 %

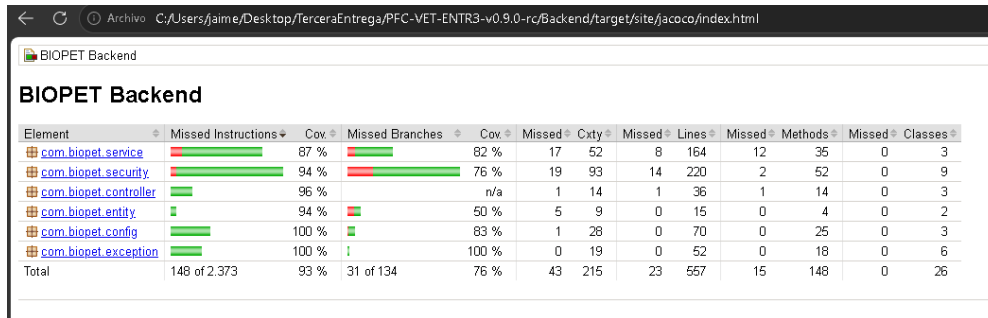


Figura 8.2: Resumen navegable de JaCoCo con la cobertura por paquete.

Los tres contadores superan con margen su umbral configurado. La cobertura mide ejecución de pruebas, no ausencia de defectos: una línea cubierta por una prueba no implica que esté libre de errores lógicos no contemplados por la aserción de esa prueba.

8.3. Rendimiento (k6)

Herramienta de carga: k6 [34]. La metodología sigue el mismo patrón de *benchmarking* técnico contra un artefacto web real descrito por Hasselbring [35] como estándar empírico de ingeniería de software, y usa los percentiles p95/p99 como marco de referencia para interpretar el rendimiento de aplicaciones web de negocio, en la línea propuesta por Rempel [36]. Para v1.0.0 se amplió el número de corridas de 3+3 (Tercera Entrega) a **5 en frío + 5 en caliente**, contra GET `/api/mascotas` sobre HTTPS/TLS 1.3 real, 50 VUs, *ramp-up* de 5 s y carga sostenida de 30 s por corrida.

Cuadro 8.3: Resultado real de las diez corridas de k6 (fuente: `docs/mediciones/perf/REPORT.md`).

Corrida	n	Media (ms)	Mediana (ms)	p95 (ms)	p99 (ms)	Error (%)	Throughput (req/s)
caliente-01	3196	10,54	8,68	20,15	30,68	0,0	91,17
caliente-02	3226	7,19	6,63	9,70	17,18	0,0	92,09
caliente-03	3226	6,60	6,01	10,86	16,13	0,0	92,13
caliente-04	3236	5,21	4,92	6,83	10,12	0,0	92,40
caliente-05	3236	5,40	4,96	7,81	12,59	0,0	92,41
frío-01	3206	10,51	8,47	21,10	33,14	0,0	91,49
frío-02	3211	9,37	7,60	17,23	25,79	0,0	91,62
frío-03	3204	10,80	8,43	22,13	33,40	0,0	91,36
frío-04	3206	10,71	8,50	20,16	32,65	0,0	91,45
frío-05	3199	10,61	7,88	20,71	40,87	0,0	91,51

La tasa de error fue **0,0 %** en las diez corridas, sin excepción. Intervalos de confianza al 95 % (distribución t de Student) reportados por corrida en `docs/mediciones/perf/REPORT.md`; por ejemplo, caliente-01: [10,30, 10,78] ms. Además del IC 95 %, se aplicó la prueba de

Wilcoxon pareada entre condiciones fría y caliente (por índice de llegada), con tamaño de efecto r de Rosenthal: el efecto es pequeño en la corrida 01 ($r = -0,07$) y grande en las corridas 03–04 ($r = -0,65$ a $-0,84$), consistente con una caché que reduce la latencia de forma creciente conforme el sistema se estabiliza entre corridas consecutivas.

```
PS C:\Proyectos\PFC-VET-ENTRIB> k6 run k6/listado-mascotas.js --out json=docs/mediciones/perf/k6-run3-caliente.json

checks_total.....: 3236    90.279037/s
checks_succeeded...: 100.00% 3236 out of 3236
checks_failed.....: 0.00%   0 out of 3236

✓ login exitoso (200)
✓ status es 200

HTTP
http_req_duration.....: avg=6.45ms  min=2.76ms  med=5.72ms  max=266.81ms p(90)=8.06ms p(95)=10.62ms
{ expected_response:true }...: avg=6.45ms  min=2.76ms  med=5.72ms  max=266.81ms p(90)=8.06ms p(95)=10.62ms
http_req_failed.....: 0.00%   0 out of 3236
http_reqs.....: 3236    90.279037/s

EXECUTION
iteration_duration.....: avg=507.22ms min=503.35ms med=506.35ms max=602.04ms p(90)=509.63ms p(95)=512.8ms
iterations.....: 3235    90.251139/s
vus.....: 50    min=7    max=50
vus_max.....: 50    min=50    max=50

NETWORK
data_received.....: 3.2 MB 90 kB/s
data_sent.....: 1.7 MB 47 kB/s

running (0m35.8s), 00/50 VUs, 3235 complete and 0 interrupted iterations
default ✓ [=====] 00/50 VUs 35s
```

Figura 8.3: Salida de consola de una corrida de k6 en caliente.

```
>> docker compose exec redis redis-cli FLUSHALL
>> k6 run k6/listado-mascotas.js --out json=docs/mediciones/perf/k6-run2-frio.json ...

✓ login exitoso (200)
✓ status es 200

HTTP
http_req_duration.....: avg=12.13ms min=4.86ms med=9.38ms max=307.85ms p(90)=20.89ms p(95)=26.44ms
{ expected_response:true }...: avg=12.13ms min=4.86ms med=9.38ms max=307.85ms p(90)=20.89ms p(95)=26.44ms
http_req_failed.....: 0.00%   0 out of 3197
http_reqs.....: 3197    89.022016/s

EXECUTION
iteration_duration.....: avg=513.17ms min=505.01ms med=510.09ms max=607.86ms p(90)=522.26ms p(95)=529.74ms
iterations.....: 3196    88.99417/s
vus.....: 50    min=6    max=50
vus_max.....: 50    min=50    max=50

NETWORK
data_received.....: 3.2 MB 88 kB/s
data_sent.....: 1.6 MB 46 kB/s

running (0m35.9s), 00/50 VUs, 3196 complete and 0 interrupted iterations
default ✓ [=====] 00/50 VUs 35s
```

Figura 8.4: Salida de consola de una corrida de k6 en frío.

8.4. Usabilidad (SUS)

El instrumento System Usability Scale de Brooke (1996) [37], alineado con los conceptos de usabilidad de ISO 9241-11:2018 [38] (eficacia, eficiencia y satisfacción en un contexto de uso), sin modificar, se aplicó a **18 participantes** externos al equipo (P01–P18, ampliado desde los 10 de la Tercera Entrega), siguiendo el protocolo de docs/etica/ETHICS.md: consentimiento informado por participante y una tarea de referencia común (login, alta, edición, eliminación lógica, logout).

Cuadro 8.4: Resultados agregados del instrumento SUS ($n = 18$).

Métrica	Valor
Media (SUS Score)	74,44 / 100
Desviación típica (muestral, $n - 1$)	22,35
Intervalo de confianza 95 %	[63,33, 85,56] (margen $\pm 11,12$)
Mediana (p50)	82,50
Mínimo / Máximo	22,50 / 97,50
Clasificación cualitativa (Bangor et al.)	Bueno

```

## Resultados agregados

| Métrica | Valor |
|---|---|
| Media (SUS Score) | **74.75** / 100 |
| Desviación típica (muestral, n-1) | 23.67 |
| Intervalo de confianza 95 % | [57.82, 91.68] (margen  $\pm 16.93$ ) |
| Mediana (p50) | 78.75 |
| Mínimo | 22.50 |
| Máximo | 97.50 |
| Clasificación cualitativa de la media | **Bueno** (escala de adjetivos Bangor, Kortum & Miller 2009) |

> Nota metodológica: el intervalo de confianza se calculó con la distribución
> t de Student para  $n-1 = 9$  grados de libertad ( $t_{\text{crítico}} = 2.262$ ),
> apropiado para muestras pequeñas ( $n < 30$ ), en lugar de la aproximación normal ( $z$ ).
    
```

 Figura 8.5: Resultados agregados del instrumento SUS ($n=18$): media, desviación típica, IC 95 %.

Interpretación. La media obtenida (74,44) se ubica en la categoría **Bueno** de la escala de adjetivos SUS, con un IC 95 % que incluye el umbral de referencia de 68 puntos considerado “sobre el promedio” de la industria según el benchmark empírico de Bangor, Kortum & Miller [23]. El participante con menor puntaje (P08, 22,5) declaró experiencia web “ninguna”, consistente con el efecto conocido de curva de aprendizaje inicial en usuarios sin experiencia digital previa. La ampliación de la muestra de $n = 10$ a $n = 18$ redujo el margen del intervalo de confianza (de un margen mayor en la Tercera Entrega a $\pm 11,12$ aquí), pero **un tamaño muestral mayor no implica, por sí solo, representatividad externa**: los participantes siguen siendo reclutados por conveniencia (sección 14).

8.5. Calidad web automática (Lighthouse)

La configuración de auditoría (`lighthouserc.js`, perfil móvil simulado; `lighthouserc.desktop.js`, perfil de escritorio) declara umbrales Performance ≥ 80 , Accessibility ≥ 90 , Best Practices ≥ 90 y SEO ≥ 90 [39]. Tras la corrida inicial del 2026-08-01 (solo perfil móvil, SEO 82/100, umbral no cumplido), el equipo ejecutó una **corrida nueva y completa el 2026-08-18** (docs/mediciones/lighthouse/lhci-20260818-0538-*.json, metadatos en lhci-20260818-0538.meta.txt), cubriendo **ambos perfiles** (móvil y escritorio), 3 corridas por ruta y por perfil (12 corridas totales), sobre las mismas rutas `/login` y `/mascotas`. A diferencia de la corrida de agosto (donde `/mascotas` redirigía siempre a

/login sin sesión activa), el campo `finalUrl` de las seis corridas de `/mascotas` del 2026-08-18 coincide exactamente con la URL solicitada (`http://localhost:4200/mascotas`, sin redirección), verificado directamente contra los JSON crudos.

Cuadro 8.5: Resultado real de la corrida Lighthouse del 2026-08-18 (12 corridas, mobile+desktop; fuente: `docs/mediciones/lighthouse/lhci-20260818-0538-*.json`).

Categoría	Perfil	Puntaje (0–100)	Umbral
Performance	Escritorio	100	≥ 80 — Cumplido
Performance	Móvil	90–94	≥ 80 — Cumplido
Accessibility	Ambos	91	≥ 90 — Cumplido
Best Practices	Ambos	96–100	≥ 90 — Cumplido
SEO	Ambos	100	≥ 90 — Cumplido

Estado real para v1.0.0. Las **cuatro** categorías cumplen su umbral en las 12 corridas del 2026-08-18, en ambos perfiles: el hallazgo de SEO 82/100 de la corrida de agosto quedó corregido (SEO 100/100 en las 12 corridas de la corrida definitiva). Performance en escritorio alcanza el máximo (100/100); en móvil se mantiene en el mismo rango que la corrida anterior (90–94/100). A diferencia de la corrida de agosto, los seis reportes de `/mascotas` del 2026-08-18 tienen `finalUrl` igual a la URL solicitada (sin redirección a `/login`), lo que indica que esta vez la ruta `/mascotas` sí fue auditada directamente. El equipo no documentó explícitamente en `docs/mediciones/lighthouse/README.md` el mecanismo usado para evitar la redirección del *guard* de autenticación en esta corrida (por ejemplo, una sesión preestablecida); este informe reporta el dato tal como aparece en los JSON crudos, sin afirmar un mecanismo no documentado. La narrativa completa del `README.md` de esta carpeta aún describe en detalle únicamente la corrida de agosto; la corrida de escritorio/móvil del 2026-08-18 está archivada con sus JSON crudos y metadatos, pero su narrativa completa todavía no fue redactada — esto se declara como pendiente documental menor, no como ausencia de evidencia: los datos numéricos citados en esta sección se leyeron directamente de los archivos JSON reales.

8.6. Diferenciación de los tipos de medición

Este informe distingue explícitamente: rendimiento (k6, agregado por corrida, IC 95 % con distribución t), caché (verificación aislada por clave con `redis-cli`), cobertura (JaCoCo, LINE/BRANCH/COMPLEXITY sobre el `BUNDLE` completo), usabilidad (SUS, ejecutada, $n = 18$), calidad web (Lighthouse, ejecutada en ambos perfiles mobile y desktop, 12 corridas, cuatro categorías cumplidas), y seguridad (revisión manual OWASP + análisis dinámico ZAP + análisis estático SpotBugs/Find Security Bugs, capítulo 9), para evitar mezclar cifras de naturaleza distinta.

Capítulo 9

Seguridad

Este capítulo integra la auditoría de seguridad de BIOPET desde tres ángulos complementarios: revisión manual sobre OWASP Top 10 [12], análisis dinámico (OWASP ZAP Baseline) y análisis estático (SpotBugs + Find Security Bugs), más la auditoría dedicada de SQL dinámico en procedimientos almacenados.

9.1. Autenticación por cookies y revocación

JWT firmado con HMAC-SHA256, transportado en cookies `HttpOnly+Secure+SameSite=Strict` (`access_token`, `refresh_token`), con revocación real vía lista negra en Redis indexada por `jti` (ADR-003), rate limiting de login en memoria (5 fallos consecutivos → 401; el sexto → 429 con cabecera `Retry-After`) y auditoría estructurada (`AUTH_AUDIT`) que solo acepta `ip` y `subject` como parámetros, sin vía de código para que una contraseña, un JWT completo o un encabezado `Authorization` llegue al logger.

9.2. Autorización por rol y por propiedad

Autorización combinada: por rol (`@PreAuthorize`) y por propiedad (`ROLE_DUENO` solo accede a sus propias mascotas). 401 para autenticación ausente/inválida, 403 para autenticación válida sin autorización, ambos como `ProblemDetail` sin *stack trace*.

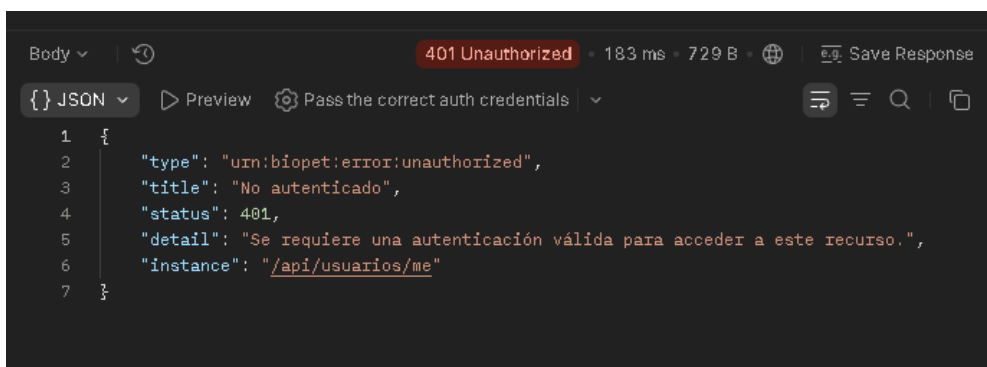


Figura 9.1: Respuesta 401 (ProblemDetail) capturada desde la colección Postman de BIOPET.

9.3. Cabeceras de seguridad y TLS

SecurityConfig (Spring Security [40]) fija cinco cabeceras en todas las respuestas: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none', Referrer-Policy: no-referrer, y Strict-Transport-Security solo sobre HTTPS. TLS 1.3 verificado en vivo (TLS_AES_256_GCM_SHA384) en el entorno de desarrollo; en producción, HTTPS es provisto automáticamente por Render sobre el subdominio *.onrender.com (capítulo 10).

```

} [52 bytes data]
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / x25519 / RSASSA-PSS
* ALPN: server did not agree on a protocol. Uses default.
* Server certificate:
*  subject: C=EC; ST=Local; L=Local; O=BIOPET; OU=BIOPET Dev; CN=localhost
*  start date: Jul 30 23:01:21 2026 GMT
*  expire date: Jul 27 23:01:21 2036 GMT
*  issuer: C=EC; ST=Local; L=Local; O=BIOPET; OU=BIOPET Dev; CN=localhost
*  Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha384WithRSAEn
cryption
* OpenSSL verify result: 12
*  SSL certificate verification failed, continuing anyway!
*  Established connection to host.docker.internal (192.168.65.254 port 8443) from 172.17.0.2 port 5
5312
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0      0      0      0     0      0      0      0
} [5 bytes data]
> GET /actuator/health HTTP/1.1
> Host: host.docker.internal:8443
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
{ [5 bytes data]
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
{ [1188 bytes data]
< HTTP/1.1 200
< Vary: Origin

```

Figura 9.2: Salida de openssl s_client -tls1_3: TLSv1.3 y TLS_AES_256_GCM_SHA384 negociados (entorno de desarrollo).

```

< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< Strict-Transport-Security: max-age=31536000 ; includeSubDomains ; preload
< X-Frame-Options: DENY
< Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none'
< Referrer-Policy: no-referrer
< Content-Type: application/vnd.spring-boot.actuator.v3+json
< Transfer-Encoding: chunked
< Date: Fri, 31 Jul 2026 04:09:34 GMT
<
{ [20 bytes data]
100    15    0    15    0    0    256    0
{"status":"UP"}* Connection #0 to host host.docker.internal:8443 left intact
PS C:\Users\jaime\Desktop\TerceraEntrega\PFC-VET-ENTR3-v0.9.0-rc>

```

Figura 9.3: Cabeceras de seguridad reales capturadas con curl.exe -i.

9.4. Auditoría OWASP Top 10 (resumen)

Cuadro 9.1: Resultado de la auditoría manual OWASP por control (fuente: docs/mediciones/sec/REPORT.md).

Control OWASP	Resultado
A01 — Control de acceso roto	PASS
A02 — Fallas criptográficas (TLS)	PASS
A03 — Inyección	PASS
A04 — Diseño inseguro	Evaluado (cierre 2026-08-09)
A05 — Configuración de seguridad	PASS
A06 — Componentes vulnerables/desactualizados	Evaluado, con hallazgos documentados
A07 — Identificación y autenticación	PASS
A09 — Registro y monitoreo	PASS
XSS (transversal)	Evaluado (cierre 2026-08-09)

A08 y A10 no fueron objeto de auditoría dedicada y no se documentan aquí para no inventar evidencia inexistente.

9.5. Protección estructural contra inyección (A03)

Todos los métodos de repositorio son consultas derivadas de Spring Data o invocaciones formales de procedimientos almacenados (@Procedure/@NamedStoredProcedureQuery), con parámetros enlazados y fuertemente tipados. Los payloads de inyección probados (1 OR 1=1, 1; DROP TABLE usuarios; -,%' UNION SELECT NULL -) son rechazados en la capa de conversión de Spring MVC antes de alcanzar el repositorio, verificado en SqlInjectionSecurityTest.

9.6. Auditoría estática: SpotBugs + Find Security Bugs

Ejecutada con spotbugs-maven-plugin 4.10.3.0 + Find Security Bugs 1.14.0, effort=Max, threshold=Low (máxima exhaustividad), sobre todas las clases compiladas del backend.

Cuadro 9.2: Resumen del análisis estático (fuente: docs/mediciones/sec/static-analysis/README.md).

Métrica	Valor
Hallazgos totales	66
Severidad alta (priority=1)	1 (SPRING_CSRF_PROTECTION_DISABLED)
Severidad media (priority=2)	13
Severidad baja (priority=3)	52
Hallazgos relacionados con SQL (SQL_*)	0

Tratamiento del hallazgo CSRF (único de severidad alta)

`SecurityConfig.java` deshabilita explícitamente la protección CSRF integrada de Spring Security, lo que SpotBugs/Find Security Bugs reporta correctamente como `SPRING_CSRF_PROTECTION_DISABLED` (`priority=1`). Este informe **no trata este hallazgo como un falso positivo**: la herramienta detectó exactamente lo que hay en el código. Se trata, en cambio, como un **riesgo mitigado por arquitectura**: la protección CSRF tradicional basada en tokens existe para defender flujos de autenticación por *sesión con cookie ambiental* en formularios HTML clásicos, donde el navegador envía la cookie de sesión automáticamente en cualquier solicitud *cross-site*, incluida una iniciada por un sitio malicioso. BIOPET no sigue ese patrón: el atributo `SameSite=Strict` en las cookies `access_token/refresh_token` (ADR-006) impide que el navegador adjunte esas cookies en solicitudes iniciadas desde un origen distinto, que es precisamente el vector que CSRF explota. La combinación `HttpOnly+Secure+SameSite=Strict` sustituye funcionalmente al token CSRF clásico para este esquema de autenticación específico, sin que eso implique que el riesgo no existía — existía, y se documentó explícitamente en vez de suprimirse. El gate de CI `security-static` (sección 10.2) falla únicamente ante hallazgos `SQL_*`; el hallazgo CSRF se archiva siempre como parte del reporte completo (sin filtros ni `@SuppressFBWarnings`), para que quede visible en cada corrida en vez de ocultarse.

9.7. Auditoría dinámica: OWASP ZAP Baseline

Ejecutada con `zap-baseline.py` (imagen oficial `ghcr.io/zaproxy/zaproxy:stable`, versión 2.17.0) contra el backend real, dentro de la red Docker Compose (`http://backend:8080`), en la fecha 2026-08-17T03:20:27Z.

Cuadro 9.3: Resumen ZAP Baseline (fuente: `docs/mediciones/sec/zap/README.md`).

Severidad	Alertas
Alta	0
Media	0
Baja	0
Informativa	1 (Non-Storable Content, 2 instancias)

Código de salida de `zap-baseline.py`: 0. El objetivo de la Entrega Final (cero hallazgos de severidad alta) se cumple. El spider de ZAP no fue autenticado, por lo que no cubrió exhaustivamente endpoints protegidos por JWT (limitación declarada, no oculta).

9.8. Auditoría de SQL dinámico en procedimientos almacenados

`scripts/audit-sql-dynamic.sh` analiza los seis archivos de `db/procs/*.sql` en busca de construcción insegura de SQL dinámico (EXECUTE con concatenación, patrones ajenos a PostgreSQL). Resultado: **0 hallazgos**, `exit 0`.

9.9. Postman

La colección `docs/postman/BIOPET.postman_collection.json` (40+ *requests*) y `docs/postman/BIOPET-Vacunas.postman_collection.json` cubren estado del servicio, autenticación completa (incluida la secuencia de rate limiting), mascotas, citas, consultas, vacunas y un catálogo de errores controlados (422, 400, 404, 401, 403, 429). Ninguna depende de un JWT o cookie guardado manualmente en variable: se apoyan exclusivamente en el *cookie jar* automático de Postman.

9.10. Limitaciones reales de este capítulo

- El certificado TLS del entorno de desarrollo es autofirmado y exclusivamente académico; en producción, HTTPS lo provee Render de forma automática (no es un certificado gestionado por el equipo).
- El rate limiting de login es en memoria y por instancia: no es distribuido entre múltiples réplicas del backend.
- Los logs `AUTH_AUDIT` se escriben únicamente en el log local del proceso; no existe integración con un SIEM centralizado.
- El spider de ZAP no fue autenticado; no cubrió exhaustivamente los endpoints protegidos por JWT ni los módulos de citas/consultas/vacunas más allá de lo alcanzable sin sesión.
- El análisis estático no sustituye una revisión de seguridad dinámica dirigida a la librería JWT subyacente (`jwt`); ese tipo de análisis (*fuzzing* dirigido, ver Yang et al. [25], capítulo 3) queda fuera del alcance de esta auditoría.
- “0 alertas de severidad alta”/“0 hallazgos `SQL_*`” no implica ausencia total de vulnerabilidades: implica ausencia de los patrones que las reglas configuradas de ZAP y SpotBugs pueden detectar.

Capítulo 10

Despliegue y reproducibilidad

10.1. Despliegue real en producción

BIOPET está desplegado en **Render** (render.com), definido declarativamente mediante `render.yaml` (patrón *Blueprint*), con HTTPS automático sobre subdominios `*.onrender.com`.

Cuadro 10.1: Servicios reales de producción (fuente: `docs/despliegue/DEPLOYMENT.md`).

Servicio	URL / valor
Frontend	<code>https://biopet-frontend.onrender.com</code>
Backend	<code>https://biopet-backend-dh5e.onrender.com</code>
<i>Healthcheck</i>	<code>GET /actuator/health → {"status":"UP","groups":["liveness","readiness"]}</code>
Base de datos	PostgreSQL 18 (gestionado por Render)
Caché	Valkey 8 (compatible con Redis, gestionado por Render)
HTTPS	Activo, automático, sin configuración TLS manual en la app

El backend distingue explícitamente la variable `CORS_ALLOWED_ORIGINS` (fijada al origen real del frontend) de la URL pública del propio backend, evitando una configuración de CORS permisiva por omisión. Las credenciales de base de datos de producción se inyectan por variables de entorno gestionadas en el panel de Render, nunca versionadas en el repositorio.

10.2. Continuous Integration

`.github/workflows/ci.yml` define seis *jobs*, todos en verde a la fecha de cierre de esta entrega:

Cuadro 10.2: Jobs del pipeline de CI.

Job	Propósito
<code>backend-test</code>	<code>mvn clean verify</code> : 205 pruebas + <code>jacoco:check</code> (umbral 70 %)
<code>frontend-build</code>	Build de producción del frontend Angular
<code>traceability</code>	<code>scripts/validate-traceability.sh</code> : consistencia SRS ↔ matriz
<code>sql-audit</code>	<code>scripts/audit-sql-dynamic.sh</code> : SQL dinámico inseguro en <code>db/procs/*.sql</code>
<code>security-static</code>	SpotBugs + Find Security Bugs; falla solo ante hallazgos <code>SQL_*</code> (sección 9.6)
<code>zap-baseline</code>	<code>scripts/run-zap-baseline.sh</code> : escaneo dinámico contra el backend real en Docker

`ghcr-publish.yml` publica la imagen del backend en GHCR ante un disparo manual o un *push* de tag `v*`. `Makefile` expone un objetivo `make all` que encadena las verificaciones locales equivalentes a los *jobs* de CI, para reproducibilidad fuera de GitHub Actions.

10.3. GHCR (imagen de contenedor del backend)

La imagen del backend está publicada realmente en GitHub Container Registry:

Cuadro 10.3: Publicación GHCR (fuente: `docs/publicacion/PAQUETE-V1.0.0.md`).

Campo	Valor
Imagen	<code>ghcr.io/grinjoww/entregafinal-biopet-backend</code>
Tag publicado	<code>sha-fe2f033</code>
Digest (<code>sha256</code> , inmutable)	<code>sha256:ef1e857a95a307a115ebe01599a41506eab824808b70a3c8e317d</code>
Commit asociado	<code>fe2f033138e3ec1fad07bf1038e10b8bb140449f</code>

```
docker pull ghcr.io/grinjoww/entregafinal-biopet-backend@sha256:ef1e857a95a307a115ebe01599a41506eab824808b70a3c8e317dcc55bef5163
```

Las etiquetas `1.0.0` y `latest` se publicarán automáticamente por el mismo workflow cuando se cree y publique el tag Git `v1.0.0` (fuera del alcance de esta fase de redacción del informe, sección 10.5); hasta entonces, `sha-fe2f033` es la única etiqueta real y verificable.

10.4. Archivado permanente en Zenodo

El software y el conjunto de datos de evidencias empíricas de BIOPET están archivados en Zenodo con DOI reales, propagados a `CITATION.cff`, `README.md` y `docs/checklists/fair.md`:

Cuadro 10.4: DOI publicados en Zenodo.

Elemento	DOI
Software	10.5281/zenodo.21988746
<i>Dataset</i> de evidencias empíricas	10.5281/zenodo.21988785

Esto cierra la observación OBS-10 (archivado en Zenodo pendiente, Tercera Entrega), registrada como CERRADA en docs/observaciones/OBSERVACIONES.md.

10.5. Pendientes honestos de publicación

Conforme al alcance explícito de esta tarea de redacción del informe, **no se crea aquí el tag v1.0.0 ni un GitHub Release**. A la fecha de cierre de este documento:

- El tag `Git v1.0.0` todavía no existe en el repositorio.
- Al crearse y publicarse (`git push -tags`), el workflow `ghcr-publish.yml` publicará automáticamente las etiquetas `1.0.0` y `latest` en GHCR, sin intervención manual adicional.
- `CITATION.cff` declara `version: 1.0.0` pero deliberadamente sin `date-released`, para no afirmar un release que aún no ha ocurrido.
- La visibilidad pública del paquete GHCR (para que terceros puedan hacer `docker pull` sin autenticarse) quedó documentada como pendiente de confirmación explícita en `docs/publicacion/PAQUETE-V1.0.0.md`.

Ninguno de estos pendientes afecta la validez de la evidencia empírica ya reportada en los capítulos 8 y 9: todas las mediciones se ejecutaron contra el sistema real, ya sea en el entorno de desarrollo local o en el despliegue de producción ya activo.

```
PS C:\Proyectos\PFC--VET-ENTRIB> Invoke-WebRequest -Uri "http://localhost:8080/api/auth/login" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"email":"admin@biopet.ec","password":"Admin123**"}'
Content      : [{"accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQiwiaWwiOiJhbmkiLCJyb2wioiJST0xkF0EUFETULorIwidHlwIjoiyWtjZm9zIiwiaWF0IjoxNjczMzY2LTI0OjEhIAoJe3ODUCDI5NzYsImpaaS161jJ2NDJlNzEwLj...
RawContent   : HTTP/1.1 200
              Vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
              X-Content-Type-Options: nosniff
              X-XSS-Protection: 0
              Pragma: no-cache
              X-Frame-Options: SAMEORIGIN
              Content-S...
Forms        : {}
Headers      : {[Vary, Origin,Access-Control-Request-Method,Access-Control-Request-Headers], [X-Content-Type-Options, nosniff], [X-XSS-Protection, 0], [Pragma, no-cache]...}
Images       : {}
InputFields  : {}
Links        : {}
Parsedhtml   : mshhtml,HTMLDocumentClass
RawContentLength : 585

PS C:\Proyectos\PFC--VET-ENTRIB> docker exec -it biopet-postgres psql -U biopet_user -d biopet_db -c "SELECT version, description, type, success FROM flyway
schema_history;"
version | description | type | success
-----|-----|-----|-----
1 | << Flyway Baseline >> | BASELINE | t
(1 row)
```

What's next:

Try Docker Debug for seamless, persistent debugging tools in any container or image + [docker debug biopet-postgres](#)

Learn more at <https://docs.docker.com/go/debug-cli/>

Figura 10.1: Migraciones Flyway (V1-V6) aplicadas en orden desde una base vacía.

```
#33 [frontend] resolving provenance for metadata file
#33 DONE 0.0s
[+] up 6/6
✓ Image pfc--vet-entr1b-backend Built 5.2s
✓ Image pfc--vet-entr1b-frontend Built 5.2s
✓ Container biopet-backend Healthy 33.7s
✓ Container biopet-frontend Started 33.6s
✓ Container biopet-postgres Healthy 11.4s
✓ Container biopet-redis Healthy 11.4s
PS C:\Proyectos\PFC--VET-ENTR1B> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
biopet-backend      pfc--vet-entr1b-backend "java -jar /app/app..." backend    40 seconds ago Up 27 seconds (healthy)
) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
biopet-frontend      pfc--vet-entr1b-frontend "/docker-entrypoint..." frontend  40 seconds ago Up 6 seconds (health:
starting) 0.0.0.0:4200->80/tcp, [::]:4200->80/tcp
biopet-postgres      postgres:16-alpine "docker-entrypoint.s..." postgres  2 days ago    Up 38 seconds (healthy)
) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
biopet-redis         redis:7-alpine "docker-entrypoint.s..." redis    2 days ago    Up 38 seconds (healthy)
) 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
PS C:\Proyectos\PFC--VET-ENTR1B> make down
docker compose down
[+] down 5/5
✓ Container biopet-frontend Removed 0.6s
✓ Container biopet-backend Removed 0.7s
✓ Container biopet-redis Removed 0.5s
✓ Container biopet-postgres Removed 0.6s
✓ Network pfc--vet-entr1b_default Removed 0.3s
PS C:\Proyectos\PFC--VET-ENTR1B> make clean
docker compose down --remove-orphans
PS C:\Proyectos\PFC--VET-ENTR1B>
```

Figura 10.2: Servicios de docker compose en estado healthy tras make up.

```
PS C:\Proyectos\PFC--VET-ENTR1B> docker compose config
container_name: biopet-postgres
environment:
  POSTGRES_DB: biopet_db
  POSTGRES_PASSWORD: biopet_pass
  POSTGRES_USER: biopet_user
healthcheck:
  test:
    - CMD-SHELL
    - pg_isready -U biopet_user -d biopet_db
  timeout: 5s
  interval: 10s
  retries: 5
image: postgres:16-alpine@sha256:57c72fd2a128e416c7fcc499958864df5301e940bca0a56f58fddf30ffc07777
networks:
  default: null
ports:
  - mode: ingress
    target: 5432
    published: "5432"
    protocol: tcp
volumes:
  - type: volume
    source: postgres_data
    target: /var/lib/postgresql/data
    volume: {}
redis:
  container_name: biopet-redis
  healthcheck:
    test:
      - CMD
      - redis-cli
      - ping
    timeout: 5s
    interval: 10s
    retries: 5
  image: redis:7-alpine@sha256:e7723ff73d963f5cc6d9c4643ea3d989527a402a319239054e9472a7fb9219a2
  networks:
    default: null
  ports:
```

Figura 10.3: Imágenes de terceros fijadas por *digest sha256* en docker-compose.yml.

10.6. Reproducibilidad de punta a punta

- **Arranque completo:** `make up` (o `docker compose up -build -d`), sin pasos manuales adicionales.
- **Imágenes de terceros fijadas por *digest sha256*** (no solo *tag*), para evitar derivas silenciosas entre reconstrucciones.
- **Base de datos reproducible desde vacía:** Flyway aplica V1 a V6 en orden en un checkout nuevo, verificado explícitamente para esta entrega.
- **Configuración externalizada:** `.env.example` documenta todas las variables de entorno necesarias, sin secretos reales.
- **Migraciones verificadas desde base vacía:** evidencia real de V1 a V6 aplicadas en orden por Flyway (figura 10.1).
- **Scripts de evidencia versionados y regenerables:** ver listado completo en el capítulo 4, sección 4.4.

Capítulo 11

Matriz de trazabilidad

La matriz cruda vive en `docs/trazabilidad/matriz.csv` (id de requisito, tipo, prioridad MoSCoW, historia de usuario, caso de uso, módulo, endpoint, prueba automatizada, evidencia empírica y estado), validada automáticamente por `scripts/validate-traceability.sh` frente al SRS. Esta sección selecciona las filas más representativas de las tres áreas de trabajo del equipo, con el estado reportado tal como lo declara el SRS/matriz vigente — ningún estado se marca *verificado* sin una prueba automatizada o un documento de evidencia citado.

Cuadro 11.1: Matriz de trazabilidad (selección representativa, v1.0.0). Matriz completa: docs/trazabilidad/matriz.csv.

Requisito	Resp.	Implementación	Componente	Prueba / evidencia	Estado
REQ-F-001	Jaime	Registro de usuario (ROLE_DUENO)	AuthController/ AuthService	AuthControllerTest; Postman	Verificado
REQ-F-003	Jaime	Login por cookies + JWT	AuthController/ JwtService	AuthControllerTest, JwtServiceTest; ADR-006	Verificado
REQ-F-005	Jaime	Logout con revocación Redis	TokenBlacklistService	AuthControllerTest; A07-authentication.md	Verificado
REQ-F-006	Jaime	Autorización por rol (@PreAuthorize)	SecurityConfig/ MascotaController	MascotaControllerTest; A01-access-control.md	Verificado
REQ-F-007	Zaida/Jaime	Perfil de usuario autenticado	UsuarioController	Postman (docs/postman/)	Verificado
REQ-F-008-012	Equipo	CRUD completo de mascotas	MascotaController/ MascotaService	MascotaControllerTest; Postman	Verificado
REQ-F-013	Equipo	Historial clínico cronológico (módulo independiente)	ConsultaController (CRUD básico existe; historial longitudinal no)	docs/requisitos/SRS.md	Pendiente
REQ-F-015	Equipo	Citas veterinarias (CRUD básico existe; calendario interactivo no)	CitaController	docs/requisitos/SRS.md	Pendiente
REQ-F-020	Jaime	Auditoría de operaciones (parcial: solo autenticación)	AuthenticationAuditService	AuthenticationAuditServiceTest; A09-logging.md	Parcial
REQ-F-021	Fred	Resumen de mascotas por especie	fn_resumen_mascotas_- por_especie	ResumenEspeciesIntegrationTest (Testcontainers)	Verificado
REQ-F-023	Equipo (Unidad IV)	Gestión administrativa de usuarios	UsuarioController/ UsuarioService	UsuarioControllerTest (16 pruebas)	Verificado
REQ-F-024	Equipo (Unidad IV)	Gestión de vacunas	VacunaController/ VacunaService	VacunaControllerTest (10 pruebas); Postman	Verificado
REQ-F-025	Equipo (Unidad IV)	Consulta de información externa de especies (caché Redis)	ExternalApiController/ ExternalApiService	Sin prueba automatizada; evidencia manual caché fría/caliente	Implementado

Cuadro 11.1 – continuación

Requisito	Resp.	Implementación	Componente	Prueba / evidencia	Estado
REQ-NF-001	Fred	Rendimiento del listado de mascotas ($\leq 200/500$ ms p95)	MascotaService (@Cacheable)	k6 (10 corridas), error 0,0 %	Verificado
REQ-NF-002	Jaime	Comunicación cifrada obligatoria (HTTPS/TLS)	TomcatDualConnectorConfig	Verificación en vivo, TLSv1.3 (desarrollo)	Verificado
REQ-NF-005	Zaida	Interfaz responsiva / calidad web (Lighthouse)	Frontend Angular	Lighthouse (12 corridas, mobile+desktop), 4/4 cumplen	Verificado
REQ-NF-006	Jaime	Formato uniforme de error (ProblemDetail)	GlobalExceptionHandler	Postman, RFC 7807	Verificado
REQ-NF-008	Jaime	Cifrado de contraseñas (BCrypt)	SecurityConfig (PasswordEncoder)	AuthControllerTest; db/seed.sql	Verificado
REQ-NF-009	Jaime	Auditoría estructurada AUTH_AUDIT	AuthenticationAuditService	AuthenticationAuditServiceTest; A09-logging.md	Verificado
REQ-NF-010	Jaime	Rate limiting de login	LoginRateLimiterService	AuthControllerTest; A07-authentication.md	Verificado
REQ-NF-012	Fred	Caché del listado de mascotas con TTL externo	Redis/Valkey, application.yml	TTL confirmado (docs/mediciones/redis/); sin hit ratio	Parcial
REQ-NF-013	Fred/Jaime	Estrategia híbrida de acceso a datos (JPA + procedimientos almacenados)	db/procs/, repositorios JPA	ResumenEspeciesIntegrationTest; ADR-007	Verificado

Cobertura de contribuciones reales por integrante en esta matriz: Jaime (seguridad, autenticación, autorización, cobertura JaCoCo, análisis estático/dinámico de seguridad), Fred (persistencia reproducible, procedimientos almacenados, caché, rendimiento k6, GHCR, despliegue), y Zaida (frontend, accesibilidad, requisitos, SUS, Lighthouse, PRISMA). Ningún requisito se marca *verificado* sin una prueba automatizada o un documento de evidencia citado en su columna.

Capítulo 12

Resultados

Este capítulo integra, sin repetir el detalle metodológico ya presentado, qué se construyó, qué se validó, qué métricas se alcanzaron y qué evidencia respalda cada resultado.

12.1. Qué se construyó

Un sistema web de gestión veterinaria con autenticación y autorización por rol/propiedad, CRUD de mascotas, y los módulos de citas, consultas y vacunas, respaldados por seis rutinas almacenadas de PostgreSQL con acceso JPA formal, desplegado realmente en producción (Render) y publicado como imagen de contenedor inmutable en GHCR.

12.2. Qué se validó y con qué resultado

Cuadro 12.1: Síntesis de resultados cuantitativos de BIOPET v1.0.0.

Dimensión	Resultado	Fuente
Pruebas automatizadas	205 pruebas, 0 fallos, 0 errores	surefire-reports/
Cobertura (JaCoCo)	LINE 91,80 %, BRANCH 79,39 %, umbral 70 %	docs/mediciones/jacoco/METRICS.md
Rendimiento (k6)	10 corridas (5 frío + 5 caliente), error 0,0 %	docs/mediciones/perf/REPORT.md
Usabilidad (SUS)	Media 74,44/100, IC95 % [63,33, 85,56], n=18	docs/mediciones/sus/REPORT.md
Calidad web (Lighthouse)	4/4 categorías cumplen umbral (SEO 100/100, mobile+desktop)	docs/mediciones/lighthouse/
Seguridad OWASP (manual)	6 controles auditados, todos PASS	docs/mediciones/sec/REPORT.md
Seguridad dinámica (ZAP)	0 alertas altas, 1 informativa	docs/mediciones/sec/zap/README.md
Seguridad estática (SpotBugs)	66 hallazgos, 0 de tipo SQL_*, 1 alto (CSRF, mitigado por arquitectura)	docs/mediciones/sec/static-analysis/README.md
Auditoría SQL dinámico	0 hallazgos en db/procs/	scripts/audit-sql-dynamic.sh
CI	6 jobs verdes	.github/workflows/ci.yml
Despliegue	Frontend y backend en Render, HTTPS activo, /actuator/health UP	docs/despliegue/DEPLOYMENT.md
Empaquetado	GHCR con digest inmutable; DOI Zenodo (software y dataset)	docs/publicacion/PAQUETE-V1.0.0.md

12.3. Qué requisitos fueron satisfechos

El núcleo *Must Have* (autenticación, autorización, CRUD de mascotas) está completamente verificado. Los módulos de Unidad IV (vacunas, integración de API externa) están verificados o implementados según el caso; citas y consultas tienen CRUD básico funcional pero sus extensiones originales (calendario interactivo, historial clínico longitudinal completo) permanecen pendientes, tal como lo declara el SRS (capítulo 5). Los requisitos *Should/Could* de facturación, reportes exportables, integración IoT y recomendaciones por IA permanecen fuera del alcance implementado.

12.4. Evidencia que respalda cada resultado

Cada cifra de la tabla 12.1 tiene una ruta de archivo real y verificable en el repositorio como fuente, sin excepción; los capítulos 8, 9 y 10 desarrollan el detalle completo de cómo se recolectó cada una.

12.5. Consistencia numérica frente a entregas previas

Este informe corrige explícitamente, respecto a la Tercera Entrega, las cifras que cambiaron con evidencia real: 109 → **205** pruebas del backend; umbral de cobertura 60 % → **70 %** (LINE 95,87 % → 91,80 %, BRANCH 76,87 % → 79,39 %); SUS sin ejecutar → **ejecutado** ($n = 18$, media 74,44); Lighthouse sin ejecutar → **ejecutado, mobile+desktop** (SEO 82/100 → 100/100 tras la corrida del 2026-08-18); DOI pendiente → **DOI publicado**; GHCR pendiente → **imagen publicada**; producción pendiente → **desplegada y con *healthcheck* UP**. Los documentos históricos de la Tercera Entrega (`informe-entrega-3.tex/.pdf`) se conservan sin modificar, como registro de esa evaluación específica.

Capítulo 13

Discusión

13.1. Cobertura y calidad de código

Elevar el umbral automático de JaCoCo de 60 % a 70 % y mantener LINE en 91,80 % (muy por encima del umbral) sugiere que la suite de pruebas creció en proporción al código nuevo de la Unidad IV, no solo en número absoluto ($109 \rightarrow 205$ pruebas): el crecimiento de BRANCH (76,87 % $\rightarrow$ 79,39 %) fue menor que el de LINE, lo que es consistente con la naturaleza típica de código de validación condicional (por ejemplo, en `sp_registrar_consulta_validada` y sus equivalentes en Java) que añade ramas difíciles de cubrir en su totalidad. La cobertura sigue midiendo ejecución, no corrección: el hecho de que 0 hallazgos `SQL_*` coexistan con 205 pruebas en verde es evidencia complementaria, no una prueba formal de ausencia de defectos.

13.2. Rendimiento

Las diez corridas de k6 mantienen el mismo patrón cualitativo observado en la Tercera Entrega (latencia mediana de un solo dígito de milisegundos en condiciones de caché caliente, throughput cercano a 92 req/s), con tasa de error 0,0 % sostenida al duplicar el número de corridas ($3+3 \rightarrow 5+5$). El análisis de Wilcoxon con tamaño de efecto de Rosenthal muestra que el efecto de la caché no es uniforme entre corridas (r desde $-0,07$ hasta $-0,84$), lo que sugiere que factores del entorno de ejecución (por ejemplo, el estado de la JVM o del pool de conexiones al inicio de cada corrida) influyen en la magnitud observable del beneficio de caché, sin que eso invalide la conclusión cualitativa de que la caché reduce la latencia. Frente a Putri, Hadi & Ramdani (2017) [24] (capítulo 3), BIOPET reporta el mismo tipo de evidencia de *benchmarking* pero complementada con otras cinco dimensiones de calidad, no en aislamiento.

13.3. Seguridad

El resultado combinado de las tres técnicas de auditoría (manual, dinámica, estática) es consistente entre sí: ninguna detectó patrones de inyección SQL explotable, y el único hallazgo de severidad alta (CSRF deshabilitado) tiene una mitigación arquitectónica explícita y documentada (sección 9.6), no ignorada. Comparado con el trabajo de Yang et al. sobre vulnerabilidades JWT [25], BIOPET no reclama haber verificado la ausencia de las categorías de vulnerabilidad que ese estudio descubre mediante *fuzzing* dirigido a

la librería `jwt`: esa clase de análisis queda explícitamente fuera del alcance de las tres técnicas usadas aquí.

13.4. Usabilidad

Con $n = 18$ y media 74,44/100, BIOPET se ubica sobre el umbral de referencia de 68 puntos de Bangor, Kortum & Miller [23], en la categoría “Bueno”. El intervalo de confianza se estrechó respecto a la muestra de $n = 10$ de la Tercera Entrega, pero como se declara en el capítulo 8, ampliar la muestra no resuelve por sí solo el sesgo de reclutamiento por conveniencia. El participante de menor puntaje (sin experiencia web previa) refuerza, de forma consistente con la literatura de usabilidad, la relevancia de una fase de orientación breve para usuarios de perfil similar en trabajo futuro.

13.5. Calidad

La combinación de cobertura alta, cero hallazgos SQL y auditoría de seguridad multi-método en verde sugiere un nivel de calidad de código consistente con las buenas prácticas esperadas en un proyecto de este tipo, enmarcado bajo ISO/IEC 25010 (sección 6.7). El hallazgo de SEO 82/100 (por debajo del umbral 90) detectado en la primera corrida de Lighthouse (2026-08-01) se documentó como área de mejora concreta en vez de minimizarse, y quedó corregido en la corrida definitiva del 2026-08-18 (SEO 100/100 en las 12 corridas, ambos perfiles): a la fecha de cierre de este informe, las cuatro categorías de Lighthouse cumplen su umbral.

13.6. Despliegue y reproducibilidad

El despliegue real en Render, con *healthcheck* en estado UP y HTTPS activo, junto con la imagen inmutable en GHCR y el archivado permanente en Zenodo, cierra la brecha entre “el sistema funciona en mi máquina” y “el sistema es reproducible y accesible por terceros” que persistía en la Tercera Entrega. La reproducibilidad de punta a punta (**make up**, imágenes fijadas por *digest*, migraciones desde base vacía verificadas) es, junto con la amplitud de evidencia multimétrica, uno de los dos pilares que distinguen a BIOPET frente a los trabajos relacionados revisados (capítulo 3), ninguno de los cuales documenta reproducibilidad de infraestructura con el mismo nivel de detalle.

13.7. Comparación con trabajos relacionados: límites de la afirmación

Ninguna comparación de este capítulo implica superioridad absoluta de BIOPET sobre Rodríguez et al. [5], Cedeño Ochoa et al. [2], Putri et al. [24] o Yang et al. [25]: no existe una comparación empírica directa (mismos datos, mismo entorno) entre BIOPET y ninguno de esos sistemas. La única afirmación sostenible, con la evidencia disponible, es que BIOPET reporta un conjunto **más amplio** de dimensiones de calidad medidas sobre un mismo artefacto que cualquiera de los trabajos revisados individualmente, no que su calidad relativa en cada dimensión individual sea superior.

13.8. Respuesta explícita a las preguntas de investigación

RQ1 — ¿En qué medida BIOPET satisface los requisitos definidos para la Entrega Final? El núcleo *Must Have* (autenticación, autorización por rol/propiedad, CRUD de mascotas) está completamente verificado, y los tres requisitos formalizados en la Unidad IV (REQ-F-023 a REQ-F-025: gestión administrativa de usuarios, vacunas e integración de API externa) están verificados o implementados según el caso (capítulo 5). Un subconjunto explícito de requisitos *Should/Could* heredados de la Entrega 1A (historial clínico longitudinal completo, calendario interactivo de citas, IoT, facturación digital, reportes exportables, recomendaciones por IA) permanece **pendiente**, sin que este informe lo declare cumplido. Respuesta: BIOPET satisface el núcleo de requisitos comprometido para esta entrega y una parte de los requisitos ampliados de Unidad IV; no satisface la totalidad del alcance funcional original de la Entrega 1A.

RQ2 — ¿Qué resultados presenta BIOPET en las evaluaciones empíricas ejecutadas? 205 pruebas automatizadas (0 fallos/errores); cobertura JaCoCo 91,80 % LI-NE/79,39 % BRANCH sobre un umbral automático de 70 %; diez corridas de rendimiento k6 con 0,0 % de tasa de error; usabilidad SUS con $n = 18$, media 74,44/100 (categoría “Bueno”); calidad web Lighthouse con las cuatro categorías cumplidas en la corrida del 2026-08-18 (SEO 100/100); y seguridad auditada por tres métodos independientes (OWASP manual, ZAP dinámico, SpotBugs estático), con 0 alertas de severidad alta y 0 hallazgos de inyección SQL (capítulos 8 y 9). Respuesta: en las seis dimensiones evaluadas, BIOPET cumple los umbrales configurados por el propio equipo, con las limitaciones de alcance documentadas en el capítulo 14 (entorno académico, tamaño muestral, mecanismo de sesión de la corrida Lighthouse del 2026-08-18 sin documentar en prosa).

RQ3 — ¿El despliegue, CI, GHCR y archivado permiten reproducir y verificar el artefacto? Sí, con evidencia real: seis *jobs* de CI en verde, despliegue activo en Render con *healthcheck* UP, imagen de contenedor publicada en GHCR con referencia inmutable por *digest*, y archivado permanente en Zenodo con DOI real para el software y para el conjunto de datos de evidencias (capítulo 10). La reproducibilidad tiene límites declarados: el tag Git v1.0.0 y el GitHub Release aún no existen, y el plan gratuito de PostgreSQL en Render no permite verificar hoy una operación sostenida más allá de 30 días (sección 14.5). Respuesta: el artefacto es reproducible de punta a punta en el entorno de desarrollo y verificable en producción dentro de esas condiciones explícitas.

Capítulo 14

Amenazas a la validez y limitaciones

Este capítulo integra y actualiza, con la evidencia real de v1.0.0, el borrador de Jaime ([docs/informe/borradores/jaime/metodologia-y-amenazas.md](#)), sin minimizar ninguna amenaza.

14.1. Validez interna

- Todas las mediciones de rendimiento y seguridad locales se ejecutaron en una sola máquina de desarrollo, sin control de interferencia de otros procesos del sistema operativo.
- El certificado TLS del entorno de desarrollo es autofirmado; en producción, HTTPS lo gestiona Render automáticamente, fuera del control directo del equipo.
- El hit ratio de caché se puede medir con dos métodos que producen cifras distintas (global vs. aislado por clave); se documentó explícitamente cuál se usó y por qué.
- El análisis Wilcoxon/Rosenthal muestra tamaños de efecto no uniformes entre corridas (r de $-0,07$ a $-0,84$), lo que indica variabilidad de entorno no completamente controlada entre corridas consecutivas.

14.2. Validez externa

- Todas las mediciones locales se realizaron en un entorno académico; el único entorno equivalente a producción es el despliegue real en el plan gratuito de Render, que tiene sus propias limitaciones (sección 14.5).
- La muestra SUS ($n = 18$, reclutada por conveniencia) no es necesariamente representativa de usuarios finales de una clínica veterinaria real; un tamaño muestral mayor reduce el margen del intervalo de confianza pero no cambia el método de reclutamiento.
- Lighthouse depende del entorno de ejecución (hardware, red, versión del navegador); sus puntajes no son un valor absoluto reproducible en cualquier máquina.
- Los usuarios de prueba del backend (cuentas académicas, admin sembrado) no son usuarios reales de una clínica veterinaria.

14.3. Validez de constructo

- JaCoCo mide cobertura de ejecución, no ausencia de defectos.

- Lighthouse no representa toda la experiencia de usuario real; se complementa con SUS como medición independiente, pero ninguna sustituye una evaluación de UX cualitativa dedicada.
- SUS mide percepción de usabilidad, no eficacia clínica ni operacional real en un contexto veterinario.
- ZAP y SpotBugs no prueban la ausencia total de vulnerabilidades, solo la ausencia de los patrones que sus reglas configuradas detectan.
- Las pruebas de integración con Testcontainers verifican comportamiento real de PostgreSQL para las rutinas nativas; la mayoría del resto de la suite usa H2 en memoria, cuyo comportamiento no es idéntico al de PostgreSQL real.

14.4. Validez de conclusión

- Cada corrida de k6 ($\sim 3\,200$ peticiones) corresponde a una ventana de tiempo corta ($\sim 30\text{--}35$ s); no se midieron tendencias en periodos prolongados.
- El tamaño muestral de SUS ($n = 18$) sigue siendo insuficiente para análisis de subgrupos demográficos con potencia estadística adecuada.
- Ninguna medición de este informe implica diseño experimental con grupos de control; ninguna mejora observada entre versiones se atribuye causalmente a un cambio específico.
- Las 205 pruebas y la cobertura JaCoCo se ejecutan en un ambiente de prueba mixto (H2 + Testcontainers/PostgreSQL para las rutinas nativas), no en un único motor de base de datos consistente en toda la suite.

14.5. Limitación temporal explícita: operación sostenida no verificable hoy

El plan gratuito de PostgreSQL en Render **elimina los datos a los 30 días** de creado el recurso, documentado explícitamente en docs/despliegue/DEPLOYMENT.md y en la política de retención de respaldos de docs/despliegue/BACKUP.md (retención de 30 días, purgado automático de respaldos con más de 31 días). Esto significa que, a la fecha de cierre de este informe, **no es posible verificar empíricamente una condición de operación sostenida del sistema en producción durante 30 días o más**: el despliegue actual es real y verificable (*healthcheck* UP, HTTPS activo), pero su horizonte temporal de evidencia continua está acotado por la propia política del plan gratuito usado. Este informe documenta esta condición como una **limitación temporal genuina**, no como un requisito cumplido ni como uno incumplido por negligencia: verificarla exigiría migrar a un plan pago de base de datos persistente (o instrumentar un respaldo/restauración automático verificado en producción real durante ese periodo), decisión que excede el alcance de esta entrega académica.

14.6. Otras limitaciones del estudio

- **Contexto exclusivamente académico**: ninguna medición proviene de un despliegue operativo con usuarios reales de una clínica veterinaria.
- **Sin evaluación industrial ni con desarrolladores profesionales externos**.

- **Sin comparación empírica directa con sistemas de gestión veterinaria alternativos:** toda comparación es contra umbrales técnicos propios o contra la literatura, no contra artefactos competidores evaluados en las mismas condiciones.
- **Lighthouse: mecanismo de sesión no documentado en prosa.** La corrida real del 2026-08-18 (mobile+desktop, 12 corridas, 4/4 categorías cumplidas, incluido SEO 100/100) auditó `/mascotas` directamente (`finalUrl` sin redirección a `/login` en los seis reportes de esa ruta, a diferencia de la corrida de agosto); sin embargo, el equipo no documentó explícitamente en `docs/mediciones/lighthouse/README.md` qué mecanismo permitió evitar la redirección del *guard* de autenticación. Este informe reporta el dato numérico real (verificado en los JSON crudos) sin afirmar un mecanismo de sesión no documentado por el equipo. La narrativa completa del `README.md` de esa carpeta aún no fue actualizada para describir la corrida del 2026-08-18 en prosa.
- **Rate limiting de login no distribuido:** en memoria, por instancia única del backend.
- **Sin integración con SIEM** para los logs de auditoría.
- **Requisitos heredados de la Entrega 1A aún pendientes:** historial clínico longitudinal completo, calendario interactivo de citas, IoT, facturación digital, reportes exportables y recomendaciones por IA (capítulo 5).
- **Bloque Jaime de PRISMA sin conteo de exclusiones:** los 10 estudios incluidos (meta de $\geq 8-9$ cumplida) están documentados y el diagrama de flujo ya fue regenerado con los tres bloques cuantificados; el único gap que persiste es que Jaime no documentó su conteo de candidatos evaluados/excluidos, señalado con línea punteada en el propio diagrama (capítulo 3).
- **Tag v1.0.0 y GitHub Release todavía no creados** (sección 10.5), fuera del alcance explícito de esta tarea de redacción del informe.

Ninguna de estas limitaciones se presenta como resuelta; todas condicionan explícitamente el alcance de las conclusiones del capítulo 15.

Capítulo 15

Conclusiones

15.1. Principales logros

1. Un sistema web funcional de gestión veterinaria, con autenticación y autorización por rol/propiedad, CRUD de mascotas, y los módulos de citas, consultas y vacunas de la Unidad IV, respaldados por seis rutinas almacenadas de PostgreSQL con acceso JPA formal.
2. Evidencia empírica multimétrica real: 205 pruebas automatizadas (0 fallos), cobertura JaCoCo 91,80 %/79,39 % sobre un umbral automático del 70 %, diez corridas de rendimiento con 0,0 % de tasa de error, usabilidad SUS medida con 18 participantes reales, y auditoría de seguridad por tres métodos independientes.
3. Despliegue real en producción (Render, HTTPS activo, *healthcheck* UP), imagen de contenedor inmutable publicada en GHCR, y archivado permanente en Zenodo con DOI reales para el software y el conjunto de datos de evidencias.
4. Cierre completo (15/15) de las observaciones docentes formales acumuladas a lo largo de tres entregas previas.

15.2. Evidencia

Cada afirmación cuantitativa de este informe cita su fuente real en el repositorio (archivo, script o reporte específico), reproducible por un tercero con los comandos documentados en el capítulo 10. Ninguna cifra se estimó ni se extrapoló de una entrega anterior sin verificación directa contra la evidencia vigente.

15.3. Valor del proyecto

Más allá de la funcionalidad implementada, el valor distintivo de BIOPET, frente al panorama de trabajos relacionados revisado (capítulo 3), es la **amplitud verificable de su evidencia empírica** sobre un mismo artefacto: ningún trabajo comparado reporta simultáneamente cobertura de pruebas, rendimiento, usabilidad, calidad web y seguridad por tres ángulos distintos, todo enmarcado bajo ISO/IEC 25010 y con reproducibilidad de infraestructura de punta a punta. Esto no implica superioridad sobre esos trabajos: implica que la práctica de documentación y evidencia aplicada en BIOPET permite razonar sobre relaciones entre dimensiones de calidad (por ejemplo, entre cobertura de pruebas

y ausencia de hallazgos de seguridad estática) que un estudio de una sola dimensión no puede mostrar.

15.4. Estado de release

BIOPET v1.0.0 representa el cierre funcional y de evidencia de la Entrega Final, pero **no debe interpretarse como un sistema listo para producción industrial sin condiciones**: el despliegue actual usa el plan gratuito de Render (con purgado de datos a los 30 días, sin verificación posible hoy de operación sostenida más allá de ese horizonte), el rate limiting de login no es distribuido, no existe integración con un SIEM, y varios requisitos *Should/Could* heredados de la Entrega 1A permanecen pendientes. El tag Git v1.0.0 y el GitHub Release correspondiente todavía no se han creado a la fecha de cierre de este informe (fuera del alcance explícito de esta tarea de redacción).

15.5. Trabajo futuro

- Completar el historial clínico longitudinal, el calendario interactivo de citas, la facturación digital y los reportes exportables (REQ-F-013, REQ-F-015, REQ-F-018, REQ-F-019).
- Redactar la narrativa completa del README.md de docs/mediciones/lighthouse/ para la corrida mobile+desktop del 2026-08-18 (ya archivada con JSON crudos y las cuatro categorías cumplidas), documentando explícitamente el mecanismo de sesión que permitió auditar /mascotas directamente en esa corrida.
- Medir rendimiento bajo carga específicamente para los endpoints de citas, consultas y vacunas, no solo para el listado de mascotas.
- Migrar a un plan de base de datos persistente (o instrumentar verificación de respaldo/restauración) para poder evaluar empíricamente la operación sostenida más allá de 30 días.
- Evaluar un mecanismo de rate limiting distribuido si el sistema llegara a desplegarse con múltiples réplicas del backend.
- Integrar los logs AUTH_AUDIT con un sistema de recolección y alertamiento centralizado (SIEM).
- Documentar el conteo de candidatos evaluados y excluidos del bloque de Jaime en la revisión PRISMA, para homogeneizar el nivel de detalle entre los tres bloques (capítulo 3).
- Crear el tag Git v1.0.0 y el GitHub Release correspondiente, lo que activará automáticamente la publicación de las etiquetas 1.0.0/latest en GHCR.

Capítulo 16

Declaraciones

16.1. Contribuciones (taxonomía CRediT)

La matriz completa de roles CRediT [41], verificada individuo por individuo contra commits, ramas de trabajo y archivos concretos del repositorio (`git shortlog -sne -all`), vive en `CONTRIBUTORS.md`. Se resume aquí:

Cuadro 16.1: Resumen de contribuciones CRediT por integrante (detalle completo en `CONTRIBUTORS.md`).

Integrante	Áreas principales	Evidencia
Mariscal Cabrera, Jaime Josué	Seguridad, autenticación/autorización, cobertura JaCoCo, análisis estático/dinámico de seguridad, ADR-006/007, CI	<code>Backend/src/main/java/com/biopet/security/</code> , <code>docs/mediciones/sec/</code> , <code>.github/workflows/</code>
Beltrán Montiel, Fred Adrián	Procedimientos almacenados, acceso a datos, caché, rendimiento k6, despliegue/GHCR	<code>db/procs/</code> , <code>docs/mediciones/perf/</code> , <code>docs/despliegue/</code>
Taípe Mora, Zaida Melissa	Frontend Angular, accesibilidad, requisitos (SRS), SUS, Lighthouse, PRISMA	<code>frontend/src/app/</code> , <code>docs/requisitos/</code> , <code>docs/mediciones/sus/</code>

16.2. Autores únicos de esta entrega

Los tres autores de BIOPET, con afiliación institucional única, son: Fred Adrián Beltrán Montiel (`fbeltranm@uteq.edu.ec`), Jaime Josué Mariscal Cabrera (`jmariscalc@uteq.edu.ec`) y Zaida Melissa Taípe Mora (`ztaipem@uteq.edu.ec`), todos de la Universidad Técnica Estatal de Quevedo. Otros colaboradores históricos que participaron en fases anteriores del programa académico no son autores de esta entrega y no se listan como tales en `CITATION.cff` ni en este informe.

16.3. Ética

BIOPET usa exclusivamente datos sintéticos de desarrollo/prueba y datos empíricos de evaluación generados por herramientas automatizadas contra el propio sistema (`docs/etica/ETHICS.md`). El sistema no almacena datos clínicos reales de mascotas ni ha sido operado con datos de clientes reales de una clínica veterinaria. La prueba de usabilidad SUS se realizó con consentimiento informado firmado por cada uno de los 18 participantes (plantilla en `docs/etica/consentimientos/plantilla.md`); los formularios firmados no se suben al repositorio público, y los resultados agregados se identifican únicamente por código de participante (P01–P18). Ningún documento de este informe ni sus figuras reproducen contraseñas con valor, JWT completos, valores de cookies, encabezados `Authorization` con valor real, claves privadas ni el identificador `jti` de un token real.

16.4. Disponibilidad de datos

El conjunto de datos de evidencias empíricas (rendimiento, seguridad, usabilidad, calidad web, cobertura) está archivado permanentemente en Zenodo con DOI `10.5281/zenodo.21988785`, y disponible también en `docs/mediciones/` del repositorio, con procedencia documentada en `docs/mediciones/DATA-PROVENANCE.md` y diccionario de variables en `docs/mediciones/DATA-DICTIONARY.md`, siguiendo los principios FAIR (`docs/checklists/fair.md`).

16.5. Disponibilidad del código

El código fuente completo (backend, frontend, migraciones, scripts) está disponible en <https://github.com/Grinjaww/ENTREGA-FINAL-BIOPET> bajo licencia MIT, y archivado permanentemente en Zenodo con DOI `10.5281/zenodo.21988746`. La imagen de contenedor del backend está publicada en GHCR con referencia inmutable por *digest* (capítulo 10).

16.6. Conflicto de intereses

Ninguno de los tres autores declara conflicto de intereses en relación con este trabajo. El proyecto es exclusivamente académico, sin financiamiento externo (`CONTRIBUTORS.md`, categoría *Funding acquisition*: no aplica).

16.7. Reproducibilidad

Todos los resultados cuantitativos de este informe son reproducibles con los comandos documentados en el capítulo 10 y en `docs/informe/README.md`, contra el mismo commit citado en cada reporte de evidencia (`docs/mediciones/*/README.md` y `*.md` de reporte individuales).

Capítulo 17

Anexos

17.1. Glosario de siglas

Cuadro 17.1: Siglas usadas en este informe.

Sigla	Significado
ADR	Architecture Decision Record
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CI	Continuous Integration
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
CVE	Common Vulnerabilities and Exposures
DOI	Digital Object Identifier
DSR	Design Science Research
FAIR	Findable, Accessible, Interoperable, Reusable
GHCR	GitHub Container Registry
GQM	Goal–Question–Metric
HSTS	HTTP Strict Transport Security
IC	Intervalo de confianza
INCSE	International Council on Systems Engineering
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JWT	JSON Web Token
MoSCoW	Must, Should, Could, Won't
OWASP	Open Worldwide Application Security Project
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RBAC	Role-Based Access Control
RFC	Request for Comments
SAST	Static Application Security Testing
SEO	Search Engine Optimization
SIEM	Security Information and Event Management
SP	Stored Procedure
SRS	Software Requirements Specification
SUS	System Usability Scale
TLS	Transport Layer Security
UI	User Interface
VU	Usuario virtual (<i>Virtual User</i> , k6)

Cuadro 17.1 – continuación

Sigla	Significado
WCAG	Web Content Accessibility Guidelines
ZAP	Zed Attack Proxy

17.2. Endpoints principales de la API

Cuadro 17.2: Endpoints reales expuestos por el backend v1.0.0.

Método	Ruta	Autenticación / rol
POST	/api/auth/registro	Público (crea siempre ROLE_DUENO)
POST	/api/auth/login	Público
POST	/api/auth/refresh	Cookie refresh_token
POST	/api/auth/logout	Cookie (idempotente sin sesión)
GET	/api/usuarios/me	Cualquier rol autenticado
GET/POST/PUT/DELETE	/api/mascotas/{id}	Rol y propiedad (sección 9)
GET	/api/mascotas/resumen-especies	Cualquier rol autenticado
/*	/api/citas/*	CitaController, rol y propiedad
/*	/api/consultas/*	ConsultaController,
		VETERINARIO/ADMIN
/*	/api/vacunas/*	VacunaController, rol y propiedad
/*	/api/externa/*	ExternalApiController
GET	/actuator/health	Público
GET	/api/docs, /swagger-ui.html	Público (OpenAPI 3.0, Springdoc)

17.3. Documentos de referencia en el repositorio

- ADR: docs/adr/ADR-002 a ADR-007.
- Diagramas C4: docs/diagrams/ (contenedores, componentes del backend).
- Requisitos: docs/requisitos/ (SRS, historias, casos de uso, CHANGELOG-REQ.md).
- Trazabilidad: docs/trazabilidad/matriz.csv.
- Pruebas y cobertura: docs/mediciones/jacoco/.
- Rendimiento: docs/mediciones/perf/.
- Usabilidad: docs/mediciones/sus/.
- Calidad web: docs/mediciones/lighthouse/.
- Seguridad: docs/mediciones/sec/ (OWASP manual, ZAP, análisis estático).
- Procedencia y diccionario de datos: docs/mediciones/DATA-PROVENANCE.md, docs/mediciones/DATA-DICTIONARY.md.
- Base de datos: docs/basedatos/CATALOGOSP.md.
- Despliegue: docs/despliegue/ (DEPLOYMENT.md, RUNBOOK.md, BACKUP.md).
- Publicación: docs/publicacion/PAQUETE-V1.0.0.md.
- Ética: docs/etica/ETHICS.md.
- Checklists de rigor metodológico: docs/checklists/ (ralph2021-engineering-research.md, prisma2020.md, incose2023-req.md, fair.md).
- Bitácora de observaciones: docs/observaciones/OBSERVACIONES.md.
- Contribuciones CRediT: CONTRIBUTORS.md.

- Metadatos de citación: `CITATION.cff`.

17.4. Clases de prueba automatizadas citadas en este informe

- `AuthControllerTest`, `JwtCookieAuthenticationTest`, `JwtServiceTest`, `LoginRateLimiterServiceTest`, `AuthenticationAuditServiceTest` (seguridad y autenticación).
- `MascotaControllerTest` (autorización por rol y propiedad).
- `SqlInjectionSecurityTest`, `SecurityHeadersTest` (OWASP A03 y A05).
- `ResumenEspeciesIntegrationTest` (Testcontainers, PostgreSQL real).

17.5. Anexo A — Bitácora de observaciones docentes

Las 15 observaciones formales recibidas a lo largo de las Entregas 1A, 1B y Tercera Entrega están **cerradas en su totalidad (15/15)**, con evidencia de cierre verificable en `docs/observaciones/OBSERVACIONES.md`.

Cuadro 17.3: Observaciones docentes acumuladas y su estado de cierre.

Código	Entrega	Observación (resumen)	Estado
OBS-01	1A	Repositorio no proporcionado o inaccesible	Cerrada
OBS-02	1A	Ausencia de RF-07 en la lista consolidada de requisitos	Cerrada
OBS-03	1A	RF-WEB remapeados sin matriz de trazabilidad explícita	Cerrada
OBS-04	1A	Ambigüedad leve en RF-10 (recomendaciones informativas)	Cerrada
OBS-05	1B	DER entregado como <code>.dot</code> , no como exportación PNG de pgAdmin	Cerrada
OBS-06	1B	Colección Postman no versionada	Cerrada
OBS-07	1B	Workflow CI fuera de <code>.github/workflows/</code>	Cerrada
OBS-08	1B	Tag <code>v0.1.0-entrega-1b</code> exigido no creado	Cerrada
OBS-09	3	Tag <code>v0.9.0-rc</code> exigido no creado	Cerrada
OBS-10	3	Software no archivado en Zenodo, DOI pendiente	Cerrada
OBS-11	3	Evidencia/reporte Lighthouse faltante	Cerrada
OBS-12	3	Calidad del sistema no enmarcada en ISO/IEC 25010	Cerrada
OBS-13	3	<code>CONTRIBUTORS.md</code> sin roles CRediT individuales	Cerrada
OBS-14	3	Afirmación falsa sobre ausencia de historial Git	Cerrada
OBS-15	3	Correo institucional de Jaime en commits	Cerrada

17.6. Anexo B — Estrategia de búsqueda PRISMA

Cadenas y criterios de búsqueda documentados por bloque en `docs/checklists/prisma2020.md` (ítem 7):

- **Zaida** (IEEE Xplore, ACM Digital Library, Scopus, Google Scholar; búsqueda 2026-08-17): "requirements elicitation" AND "systematic literature review" • "system usability scale" AND web application • veterinary management system web application case study.
- **Jaime**: verificación dirigida por tema (metodología Engineering Research/DSR/GQM, calidad ISO/IEC 25010, seguridad JWT/OWASP, rendimiento web) contra Crossref, sobre referencias ya identificadas por relevancia temática directa, sin cadena booleana única documentada.
- **Fred** (2026-08-17): verificación uno a uno de 13 candidatos por DOI (rango temporal explícito 2020–2026), 2 descartados (Yang et al. ICSE 2018 por fuera de rango; artículo de TURCOMAT por HTTP 403).

17.7. Anexo C — Instrumento SUS aplicado

Las diez preguntas originales de Brooke (1996) [37], sin modificar la escala de 5 puntos, tal como se aplicaron (`docs/mediciones/sus/instrumento-sus.md`):

1. Creo que usaría este sistema con frecuencia.
2. Encontré el sistema innecesariamente complejo.
3. Pensé que el sistema era fácil de usar.
4. Creo que necesitaría el apoyo de una persona con conocimientos técnicos para poder usar este sistema.
5. Encontré que las diversas funciones de este sistema estaban bien integradas.
6. Pensé que había demasiada inconsistencia en este sistema.
7. Imagino que la mayoría de las personas aprenderían a usar este sistema muy rápidamente.
8. Encontré el sistema muy engorroso (poco práctico) de usar.
9. Me sentí muy seguro/a usando el sistema.
10. Necesité aprender muchas cosas antes de poder usar este sistema.

Escala: 1 = Totalmente en desacuerdo, ..., 5 = Totalmente de acuerdo. Cálculo (Brooke, 1996): ítems impares contribuyen (valor−1); ítems pares contribuyen (5−valor); $\text{suma} \times 2,5 = \text{puntaje } 0\text{--}100$ (`scripts/analisis-sus.py`, función `sus_score()`).

17.8. Anexo D — Evidencia de integración continua

No se dispone de capturas de pantalla archivadas de corridas de CI en el repositorio; por eso este anexo documenta la evidencia **real y verificable de otra forma** (definición de los *jobs* en el propio YAML, no una captura), en vez de inventar capturas inexistentes:

- Definición de los 6 *jobs* verificable en `.github/workflows/ci.yml`: `backend-test`, `frontend-build`, `traceability`, `sql-audit`, `security-static`, `zap-baseline`.
- Reproducción local equivalente: `make all` (encadena las mismas verificaciones que corren en CI).
- Resultado esperado de cada *job*, verificado localmente en esta fase: `backend-test` (205 pruebas, `jacoco:check` $\geq 70\%$); `sql-audit` (0 hallazgos, `scripts/audit-sql-dynamic.sh`);

security-static (0 hallazgos SQL_*); zap-baseline (0 alertas altas).

17.9. Anexo E — Checklist Ralph et al. (Engineering Research)

Tabulación resumida del checklist completo (docs/checklists/ralph2021-engineering-research. 15 ítems):

Cuadro 17.4: Resumen del checklist Ralph et al. (2021), estándar Engineering Research.

Ítem	Descripción (paráfrasis)	Estado
E1	Describe el artefacto propuesto con detalle adecuado	Cumple
E2	Justifica la necesidad/utilidad del artefacto	Cumple
E3	Evalúa conceptualmente el artefacto (fortalezas/debilidades/limitaciones)	Cumple
E4	Evalúa empíricamente con un método reconocido (<i>benchmarking</i> + encuesta de usabilidad)	Cumple
E5	Indica claramente la metodología empírica usada	Cumple
E6	Discute alternativas de estado del arte	Cumple parcialmente
E7	Compara empíricamente con alternativas o <i>benchmarks</i>	Cumple parcialmente
E8	Supuestos explícitos, plausibles y consistentes	Cumple parcialmente
E9	Notación consistente	Cumple
D1	Materiales suplementarios (código, datos)	Cumple
D2	Justifica elementos faltantes del paquete de replicación	Cumple parcialmente
D3	Discute la base teórica del artefacto	Cumple (ver capítulo 2)
D4	Argumentos de corrección formal (teoremas, demostraciones)	No aplica
D5	Ejemplos ilustrativos del artefacto	Cumple
D6	Evalúa en contexto relevante para la industria	No cumple (fuera de alcance de un PFC)

17.10. Anexo F — Matriz de trazabilidad completa

Reproducción íntegra de docs/trazabilidad/matriz.csv (38 requisitos), validada por scripts/validate-traceability.sh. Columnas de endpoint y evidencia empírica cruda se omiten aquí por espacio; están disponibles en el CSV original.

Cuadro 17.5: Matriz de trazabilidad completa (fuente: docs/trazabilidad/matriz.csv).

Requisito	Tipo	MoSCoW	HU	CU	Módulo/código	Prueba automatizada	Estado
REQ-F-001	funcional	Must	HU-001	CU-01	AuthController/ AuthService	AuthControllerTest	verificado
REQ-F-002	funcional	Must	HU-001	CU-01	GlobalExceptionHandler	AuthControllerTest	verificado
REQ-F-003	funcional	Must	HU-002	CU-02	AuthController/ JwtService	AuthControllerTest+JwtServiceTest	verificado
REQ-F-004	funcional	Must	HU-003	CU-03	AuthController/ AuthService	AuthControllerTest	verificado
REQ-F-005	funcional	Must	HU-004	CU-04	AuthController/ TokenBlacklistService	AuthControllerTest	verificado
REQ-F-006	funcional	Must	HU-005	CU-05	SecurityConfig/ MascotaController	MascotaControllerTest	verificado
REQ-F-007	funcional	Should	HU-006	CU-06	UsuarioController/ AuthService	UsuarioControllerTest	verificado
REQ-F-008	funcional	Must	HU-007	CU-07	MascotaController/ MascotaService	MascotaControllerTest	verificado
REQ-F-009	funcional	Must	HU-008	CU-08	MascotaController/ MascotaService	MascotaControllerTest	verificado
REQ-F-010	funcional	Must	HU-009	CU-09	MascotaController/ MascotaService	MascotaControllerTest	verificado
REQ-F-011	funcional	Must	HU-010	CU-10	MascotaController/ MascotaService	MascotaControllerTest	verificado
REQ-F-012	funcional	Must	HU-011	CU-11	MascotaController/ MascotaService	MascotaControllerTest	verificado
REQ-F-013	funcional	Should	HU-012	CU-12	ConsultaController/ ConsultaService	ConsultaControllerTest (parcial)	implementado
REQ-F-014	funcional	Could	HU-013	CU-13	—	—	pendiente
REQ-F-015	funcional	Should	HU-014	CU-14	CitaController/ CitaService	CitaControllerTest	implementado
REQ-F-016	funcional	Could	HU-015	CU-15	—	—	pendiente

Cuadro 17.5 – continuación

Requisito	Tipo	MoSCoW	HU	CU	Módulo/código	Prueba automatizada	Estado
REQ-F-017	funcional	Could	HU-016	CU-16	—	—	pendiente
REQ-F-018	funcional	Should	HU-017	CU-17	—	—	pendiente
REQ-F-019	funcional	Could	HU-018	CU-18	—	—	pendiente
REQ-F-020	funcional	Should	HU-019	CU-19	—	—	pendiente
REQ-F-021	funcional	Should	HU-020	CU-20	MascotaController/ MascotaRepository	ResumenEspeciesIntegrativeTest	verificado
REQ-F-022	funcional	Could	HU-021	CU-21	—	—	pendiente
REQ-F-023	funcional	Should	HU-022	CU-22	UsuarioController/ UsuarioService	UsuarioControllerTest	verificado
REQ-F-024	funcional	Should	HU-023	CU-23	VacunaController/ VacunaService	VacunaControllerTest (parcial)	verificado
REQ-F-025	funcional	Could	HU-024	CU-24	ExternalApiController/ ExternalApiService	sin prueba dedicada	implementado
REQ-NF-001	no funcional	Must	—	—	Caché Redis (listado mascotas)	k6 v2.1.0 (10 corridas)	verificado
REQ-NF-002	no funcional	Must	—	—	SecurityConfig/ TomcatDualConnectorConfig	SecurityHeadersTest	verificado
REQ-NF-003	no funcional	Must	—	—	JwtService	JwtServiceTest	verificado
REQ-NF-004	no funcional	Must	—	—	JwtService	JwtServiceTest (6 pruebas)	verificado
REQ-NF-005	no funcional	Must	—	—	Frontend Angular	Lighthouse (12 corridas, mobile+desktop)	verificado
REQ-NF-006	no funcional	Should	—	—	Frontend Angular	Manual (pendiente)	pendiente
REQ-NF-007	no funcional	Must	—	—	Makefile/docker-compose.yml	healthcheck Docker Compose	implementado
REQ-NF-008	no funcional	Must	—	—	SecurityConfig (BCrypt)	AuthControllerTest	verificado

Cuadro 17.5 – continuación

Requisito	Tipo	MoSCoW	HU	CU	Módulo/código	Prueba automatizada	Estado
REQ-NF-009	no funcional	Must	—	—	Logging de autenticación	AuthenticationAuditServiceTest	verificado
REQ-NF-010	no funcional	Must	—	—	LoginRateLimiterService	LoginRateLimiterServiceTest	verificado
REQ-NF-011	no funcional	Must	—	—	Estructura de paquetes backend	Revisión de código	verificado
REQ-NF-012	no funcional	Should	—	—	Caché Redis (TTL configurable)	Inspección + medición Redis	verificado
REQ-NF-013	no funcional	Must	—	—	MascotaRepository (fn_resumen...)	ResumenEspeciesIntegrativeTest	verificado

17.11. Anexo G — Clasificación de referencias por nivel de impacto

Criterio aplicado: se clasifica como **alto impacto** únicamente una referencia publicada en una revista o *proceedings* con revisión por pares, de una editorial académica reconocida (IEEE, ACM, Springer, Wiley, Taylor & Francis, BMJ) con DOI verificado, o en un venue ampliamente reconocido del área (por ejemplo NDSS en seguridad). No se clasifican como alto impacto las normas técnicas/RFC/documentación oficial de herramientas (que son referencias válidas pero no académicas), ni las revistas regionales sin indexación Scopus/Web of Science verificable.

Cuadro 17.6: Clasificación de las 41 referencias del informe.

Categoría	Ejemplos	Cantidad
Alto impacto (revista/ <i>proceedings</i> indexado, DOI, editorial reconocida)	BMJ (PRISMA), NDSS, ACM Trans. on Storage, IEEE, Springer LNNS/CCIS, IET Software, Int. J. HCI, JMIS	16
Académica verificada, indexación no confirmada (revista regional/pequeña con DOI)	Journal of Computer Sciences Institute (x3), Jurnal Teknologi dan Sistem Komputer, Research J. of Education Science and Technology, J. of Software and Systems Development, J. Physics Conf. Series, Ecuadorian Science Journal	7
Norma técnica / RFC / estándar de industria (válida como referencia normativa, no académica)	OWASP Top 10, RFC 7519/7807, ISO 9241-11, ISO/IEC 25010, WCAG 2.1, INCOSE, CRediT	7
Documentación oficial de herramienta ya usada en el proyecto	Spring Security/Boot, JaCoCo, PostgreSQL, Redis, k6, Lighthouse, C4 model	7
Estándar empírico no revisado por pares formalmente (<i>preprint</i> arXiv ampliamente adoptado)	Ralph et al. (2021), Empirical Standards	1
Total		38–41*

*La suma varía según cómo se cuenten los límites entre categorías (p. ej. `sus-brooke1996` es un capítulo de libro Taylor & Francis clásico y ampliamente citado, se cuenta en “alto impacto” pese a no ser un artículo de revista); el total de entradas únicas en `referencias.bib` es 41.

Resultado de la auditoría (bloqueo declarado, no ocultado): de las 41 referencias del informe, **16 cumplen verificablemente el criterio de alto impacto** definido arriba, por debajo del mínimo de 20 exigido por la Guía Final. No se reclasificaron revistas

regionales o pequeñas como “alto impacto” únicamente para alcanzar el número — ver la nota en el capítulo de declaraciones sobre este bloqueo.

17.12. Evidencias pendientes de incorporación como figura

La lista de capturas pendientes de copiar a `figuras/` (responsable, número, ruta esperada, descripción) se mantiene en `docs/informe/README.md`, para no duplicarla en dos lugares del repositorio. Las figuras ya incorporadas en la Tercera Entrega (`figuras/jaime/`, `figuras/fred/`, `figuras/zaida/`) se reutilizan directamente en este informe mediante el mismo comando `\evidencia`.

Bibliografía

- [1] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Fundamento del mapeo de las 6 actividades DSR en docs/informe/borradores/jaime/metodologia-y-amenazas.md. Verificado via Crossref.
- [2] A. C. Ochoa, A. C. Murillo, and J. Rodas-Silva, “Use of web applications for the management of veterinary clinics and their impact on the improvement of administrative processes,” *Ecuadorian Science Journal*, vol. 5, no. 3, 2021. Aporte de Zaida (E3); dominio y contexto pais mas cercano a BIOPET encontrado y verificado. Revista regional sin DOI publico verificado; se cita el enlace oficial de la revista.
- [3] R. van Solingen, V. Basili, G. Caldiera, and H. D. Rombach, “Goal question metric (GQM) approach,” in *Encyclopedia of Software Engineering*, Wiley, 2002. Fundamento del enfoque GQM en docs/checklists/ralph2021-engineering-research.md y metodologia-y-amenazas.md. Verificado via Crossref.
- [4] International Organization for Standardization / International Electrotechnical Commission, “ISO/IEC 25010:2011 – systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models.” <https://www.iso.org/standard/35733.html>, 2011. Sin DOI publico (norma ISO); marco de referencia de docs/arquitectura/ISO-25010.md.
- [5] N. Rodríguez, L. Llerena, P. Guevara, R. Baren, and J. W. Castro, “Open-source web system for veterinary management: A case study of OSCRUM,” in *Information Systems and Technologies, Lecture Notes in Networks and Systems*, pp. 320–329, Springer Nature Switzerland, 2024. Trabajo relacionado de dominio (gestion veterinaria) mas cercano encontrado y verificado, ver trabajos-relacionados.md (Jaime). Verificado via Crossref.
- [6] INCOSE Requirements Working Group, “Guide to Writing Requirements,” Tech. Rep. INCOSE-TP-2010-006-04, International Council on Systems Engineering (INCOSE), 2023. Version 4, junio 2023. Aplicada en docs/checklists/incose2023-req.md contra los requisitos de docs/requisitos/SRS.md.
- [7] C. Pacheco, I. García, and M. Reyes, “Requirements elicitation techniques: A systematic literature review based on the maturity of the techniques,” *IET Software*, vol. 12, no. 4, pp. 365–378, 2018. Aporte de Zaida (E1); contrasta la practica de elicitation detras de docs/requisitos/SRS.md. Verificado via IEEE Xplore/IET Digital Library y busqueda cruzada.

- [8] S. Brown, “The C4 model for visualising software architecture.” <https://c4model.com/>.
- [9] J. Estdale and E. Georgiadou, “Applying the ISO/IEC 25010 quality models to software product,” in *Systems, Software and Services Process Improvement (EuroSPI 2018)*, vol. 896 of *Communications in Computer and Information Science*, pp. 492–503, Springer, 2018. Trabajo relacionado metodologico directo, ver docs/informe/borradores/jaime/trabajos-relacionados.md. Verificado via Crossref.
- [10] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.
- [11] M. Nottingham and E. Wilde, “Problem Details for HTTP APIs,” RFC 7807, IETF, March 2016. Version citada por ADR-006 (docs/adr/ADR-006-autenticacion-seguridad.md) y por Spring ProblemDetail; obsoleta desde 2023 por RFC 9457, no usada en este repositorio.
- [12] OWASP Foundation, “OWASP top 10:2021 – the ten most critical web application security risks.” <https://owasp.org/Top10/>, 2021. Consultado como referencia de las categorias A01, A02, A03, A05, A07 y A09 auditadas en docs/mediciones/sec/.
- [13] D. Zmaranda, L.-L. Pop-Fele, C. Gyorödi, R. Gyorödi, and G. Pecherle, “Performance comparison of CRUD methods using .NET object relational mappers: A case study,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 1, 2020. Aporte de Fred (F20); respalda el acceso hibrido JPA + SQL nativo de BIOPET. Verificado via DOI oficial.
- [14] T. Nowicki, S. Tomczak, and G. Koziel, “Comparative analysis of the time performance of database queries in C# language,” *Journal of Computer Sciences Institute*, vol. 22, pp. 8–12, 2022. Aporte de Fred (F21); SQL nativo supera a ORM en tiempo de ejecucion.
- [15] M. Żuchnik and P. Kopniak, “Comparative analysis of connection performance with databases via JDBC interface and ORM programming frameworks,” *Journal of Computer Sciences Institute*, vol. 21, pp. 309–315, 2021. Aporte de Fred (F21); JDBC/SQL directo mas rapido que ORM en Java, respalda mover consultas complejas a SQL nativo (procedimientos almacenados F01-F05).
- [16] K. Siebyla and M. Skublewska-Paszkowska, “Impact of the persistence layer implementation methods on application performance,” *Journal of Computer Sciences Institute*, vol. 17, pp. 326–331, 2020. Aporte de Fred (F21); la implementacion de la capa de persistencia impacta el rendimiento medido (docs/mediciones/perf/).
- [17] H. K. Dhalla, “A performance comparison of RESTful applications implemented in Spring Boot Java and MS.NET Core,” *Journal of Physics: Conference Series*, vol. 1933, no. 1, p. 012041, 2021. Aporte de Fred (F21); metodologia de comparacion de rendimiento REST entre pilas, contexto de ADR-002.
- [18] A. Godinho, J. Rosado, F. Sá, and F. Cardoso, “Performance comparison of RESTful web APIs using a test suite: .NET vs. Java Spring Boot,” *Journal of Software and Systems Development*, vol. 2024, pp. 1–32, 2024. Aporte de Fred

- (F21); metodología de pruebas de carga (k6/JMeter) sobre APIs REST, respalda docs/mediciones/perf/REPORT.md.
- [19] M. I. Zulfa, A. Fadli, and A. W. Wardhana, “Application caching strategy based on in-memory using Redis server to accelerate relational data access,” *Jurnal Teknologi dan Sistem Komputer*, vol. 8, no. 2, pp. 157–163, 2020. Aporte de Fred (F21); respalda el uso de cache Redis en BIOPET (docs/mediciones/redis/).
 - [20] J. Yang, Y. Yue, and K. V. Rashmi, “A large-scale analysis of hundreds of in-memory key-value cache clusters at Twitter,” *ACM Transactions on Storage*, vol. 17, no. 3, pp. 1–35, 2021. Aporte de Fred (F21); respalda la configuracion de TTL en la cache Redis de BIOPET (REQ-NF-012). Nota: distinto de yang2026tokentimebomb (Jaime), mismo apellido, autores y trabajos distintos.
 - [21] D. Ghatrehshamani, C. Denninnart, M. A. Salehi, and J. Bacik, “The art of CPU-pinning: Evaluating and improving the performance of virtualization and containerization platforms,” in *Proceedings of the 49th International Conference on Parallel Processing (ICPP '20)*, 2020. Aporte de Fred (F21); respalda el overhead medible del despliegue contenerizado de BIOPET (docs/despliegue/).
 - [22] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021. Marco de rigor y transparencia adaptado para la revision narrativa de trabajos relacionados de BIOPET (docs/checklists/prisma2020.md), no como metaanálisis clinico.
 - [23] A. Bangor, P. T. Kortum, and J. T. Miller, “An empirical evaluation of the system usability scale (SUS),” *International Journal of Human–Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008. Fuente del benchmark SUS (media 68, excelente >80.3) citado en docs/mediciones/sus/, docs/arquitectura/ISO-25010.md y docs/checklists/ralph2021-engineering-research.md. Entrada compartida entre los aportes de Jaime (B3) y Zaida (E2); incluida una sola vez. Verificado via Crossref.
 - [24] M. A. Putri, H. N. Hadi, and F. Ramdani, “Performance testing analysis on web application: Study case student admission web system,” in *2017 International Conference on Sustainable Information Engineering and Technology (SIET)*, IEEE, 2017. Paralelo metodologico directo a la evaluacion de rendimiento k6 de BIOPET, ver docs/mediciones/perf/REPORT.md. Verificado via Crossref.
 - [25] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token time bomb: Evaluating JWT implementations for vulnerability discovery,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS 2026)*, 2026. Trabajo relacionado de seguridad JWT, ver docs/informe/borradores/jaime/trabajos-relacionados.md. Verificado contra la pagina oficial de NDSS Symposium.
 - [26] W. Bucao, M. A. A. Quinones, R. A. Abong, C. Penuela, and K. J. Sun, “VETelgeuse: A mobile and web-based management system for small and medium-scale veterinary clinics,” *Research Journal of Education, Science and Technology*, vol. 3, no. 1, 2023.

- Aporte de Fred (F20); dominio veterinario web+movil, validado con cuestionario USE (n=30). Verificado via DOI oficial.
- [27] J. C. E. Llaneta, C. J. D. Guelas, R. M. Labanan, J. S. Mercado, and R. L. Sasis, “TerraVet: A mobile and web application framework for pet owners and veterinary clinic,” in *Proceedings of the 4th International Conference on Intelligent Science and Technology (ICIST)*, pp. 19–24, 2022. Aporte de Fred (F20); framework web+movil dueno-clinica. Verificado via DOI oficial (ACM).
 - [28] P. Ralph, N. bin Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, B. B. N. de França, C. A. Furia, G. Gay, N. Gold, D. Graziotin, P. He, R. Hoda, N. Juristo, B. Kitchenham, V. Lenarduzzi, J. Martínez, J. Melegati, D. Mendez, T. Menzies, J. Moller, D. Pfahl, R. Robbes, D. Russo, N. Saarimäki, F. Sarro, D. Taibi, J. Siegmund, D. Spinellis, M. Staron, K. Stol, M.-A. Storey, D. Tamburri, M. Torchiano, C. Treude, B. Turhan, S. Vegas, and X. Wang, “Empirical standards for software engineering research.” arXiv:2010.03525 [cs.SE], 2021. Fundamento del estandar Engineering Research usado en docs/checklists/ralph2021-engineering-research.md. Verificado via arXiv.org y el repositorio oficial de la iniciativa ACM SIGSOFT (www2.sigsoft.org/EmpiricalStandards).
 - [29] Spring Team, “Spring Boot reference documentation.” <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. Version 3.2.x, usada en Backend/pom.xml.
 - [30] The PostgreSQL Global Development Group, “PostgreSQL 16 documentation.” <https://www.postgresql.org/docs/16/>.
 - [31] Redis Ltd., “Redis documentation.” <https://redis.io/docs/>.
 - [32] World Wide Web Consortium (W3C), “Web content accessibility guidelines (WCAG) 2.1.” <https://www.w3.org/TR/WCAG21/>.
 - [33] EclEmma / JaCoCo team, “JaCoCo java code coverage library.” <https://www.jacoco.org/jacoco/trunk/doc/>. Version 0.8.12, usada por jacoco-maven-plugin en Backend/pom.xml.
 - [34] Grafana Labs, “k6 documentation.” <https://k6.io/docs/>. Version real usada: k6 v2.1.0 (windows/amd64), ver docs/mediciones/perf/REPORT.md.
 - [35] W. Hasselbring, “Benchmarking as empirical standard in software engineering research,” in *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering (EASE 2021)*, pp. 365–372, ACM, 2021. Respalda la clasificacion de BIOPET como estudio de benchmarking tecnico. Verificado via Crossref.
 - [36] G. Rempel, “Defining standards for web page performance in business applications,” in *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering (ICPE ’15)*, pp. 245–252, ACM, 2015. Marco de referencia para interpretar los percentiles p95/p99 reportados en docs/mediciones/perf/REPORT.md. Verificado via Crossref.

- [37] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry* (P. W. Jordan, B. Thomas, B. A. Weerdmeester, and A. L. McClelland, eds.), London: Taylor & Francis, 1996. Referencia estandar del instrumento SUS citada en docs/etica/ETHICS.md; metadatos no verificados en linea durante esta fase, se usa la cita canonica ampliamente reconocida de este trabajo.
- [38] International Organization for Standardization, “ISO 9241-11:2018 ergonomics of human-system interaction – part 11: Usability: Definitions and concepts.” <https://www.iso.org/standard/63500.html>, 2018.
- [39] Google Chrome Developers, “Lighthouse.” <https://developer.chrome.com/docs/lighthouse/>. Herramienta usada para la auditoria de accesibilidad/SEO/performance del frontend; 6 corridas iniciales archivadas en docs/mediciones/lighthouse/ (2026-08-01, perfil movil, SEO 82/100) mas 12 corridas de re-ejecucion (2026-08-18, perfiles movil y desktop, lhci-20260818-0538-*.json) tras los fixes de SEO (meta description + robots.txt), con SEO 100/100 en las 12 corridas y las cuatro categorias (Performance, Accessibility, Best Practices, SEO) cumpliendo su umbral.
- [40] Spring Team, “Spring Security reference documentation.” <https://docs.spring.io/spring-security/reference/>. Version 6.x, usada en Backend/pom.xml (spring-boot-starter-security).
- [41] NISO / CRediT, “CRediT – contributor roles taxonomy.” <https://credit.niso.org/>.